

Machine Learning- Based Detection of Network Intrusion Traffic

TU885

Csaba Palosi
10/05/2026

TABLE OF CONTENTS

Abstract	3
Chapter 1: Introduction	4
1.1 Background	4
1.2 Problem Definition.....	5
1.3 Aim.....	5
1.4 Objectives	6
1.5 Hypothesis.....	6
1.6 Research Questions	6
1.7 Scope of the Study	7
Chapter 2: Literature Review	7
2.1 Introduction.....	7
2.2 Intrusion Detection Systems	7
2.3 Machine Learning in intrusion detection systems (IDS)	8
2.4 Network Traffic Datasets	9
2.5 Class Imbalance Problem.....	9
2.6 SMOTE and Borderline-SMOTE	10
2.7 Related Work.....	10
2.8 Research Gap	11
2.9 Summary	11
Chapter 3: Methodology.....	12
3.1 Overview	12
3.2 Dataset and Understanding	13
3.3 Data Preprocessing.....	16
3.4 Feature Selection.....	20
3.5 Model Selection	21
3.6 Class Imbalance Handling	22
3.7 Train-Test Split Strategy	26
3.8 Evaluation Metrics	26
3.9 Machine Learning Pipeline	28
3.10 Computational Environment.....	30

Chapter 4: Experiments and Results	31
4.1 Experimental Setup	31
4.2 Baseline Configuration (Imbalanced Dataset)	32
4.3 SMOTE Results	35
4.4 Borderline-SMOTE Results.....	41
4.5 Experimental Results Comparison.....	47
4.6 Discussion	51
Chapter 5: Conclusion.....	52
5.1 Summary of the Research	52
5.2 Achievement of Objectives	52
5.3 Research Questions Answered	52
5.4 Hypothesis Evaluation	53
5.5 Contribution to the Field.....	53
5.6 Final Remarks	54
Chapter 6: Future Work.....	55
6.1 Real-Time Deployment in Virtual Lab Environments	55
6.2 Development of a Real-Time Intrusion Detection System	55
6.3 Feature Engineering Improvements	55
6.4 Exploration of Advanced Machine Learning Models	56
6.5 Evaluation on Additional Datasets	56
6.6 Summary	56
References	57
Appendix	61

Abstract

Machine learning-based intrusion detection systems have become increasingly important in identifying malicious network activity in modern cybersecurity environments. Traditional signature-based approaches struggle to detect evolving and previously unseen attack patterns, particularly in large-scale, imbalanced network traffic datasets.

This study investigates the application of a Random Forest classifier for multi-class network intrusion detection using the CICIDS2018 dataset focusing on its manually selected merged subset. The original dataset includes benign traffic and multiple attack categories such as botnet activity, brute-force attacks, denial-of-service (DoS), and distributed denial-of-service (DDoS) traffic. A comprehensive preprocessing pipeline was configured to be able to handle missing values, duplicate removal, and select appropriate features based on network flow characteristics.

To solve the significant class imbalance found in the dataset, Synthetic Minority Oversampling Technique (SMOTE) and Borderline-SMOTE were used to the training data. Furthermore, hyperparameter tuning of the Random Forest classifier was also applied on both technique (SMOTE, Borderline-SMOTE) to improve overall robustness and performance.

The configurations were evaluated using accuracy, precision, recall, F1-score, and macro-averaged F1-score to ensure balanced performance assessment across all classes.

The results show that the applied Random Forest classifier greatly benefited from both class imbalance handling and hyperparameter optimisation. The best-performing was from the tested 5 configuration (including the baseline version) was the SMOTE-based one combined it with the hyperparameter tuning technique, which got the highest macro F1-score of **0.8154** and an accuracy of **0.9628**, outperforming both the baseline which got a macro F1-score of **0.0865** and an accuracy of **0.3285** and the Borderline-SMOTE variant. While Borderline-SMOTE improved class boundary learning in certain cases, it did not get any better macro F1-score and overall performance compared to SMOTE.

The found results demonstrate that addressing class imbalance and tuning parameters are both critical for improving the reliability of machine learning-based on intrusion detection systems. This research highlights how effective can be of ensemble learning combination with resampling techniques and parameter optimisation in detecting network attacks and also

provides a foundation for future work in real-time intrusion detection and explainable artificial intelligence.

Chapter 1: Introduction

1.1 Background

Cybersecurity become a critical concern in the past few years as modern networks generate large volumes of complex and heterogeneous traffic (Abbasi, Shahraki and Taherkordi, 2021). These refer to different categories of cyberattacks that aim to disrupt services, exhaust system resources, or gain unauthorized access to networks and systems. Malicious activities such as denial-of-service (DoS), distributed denial-of-service (DDoS), brute-force attacks, and botnet infections just to name a few can be a difficult challenge to detect from the everyday user to big international companies or even government entities especially when those are embedded within legitimate network communication. The traditional signature-based Intrusion Detection Systems or IDS rely on predefined rules and known attack patterns, what is limits their ability to identify novel or evolving threats (Hnamte and Hussain, 2023).

The result for that is a significant move from the old technique toward to the data-driven approaches that utilise machine learning techniques to model normal and malicious behaviour based on network traffic characteristics (Garcia-Teodoro et al., 2009; Liao et al., 2013; Buczak and Guven, 2016; Sarker et al., 2020). These approaches let any kind of systems to learn and identify patterns directly from the actual data instead of relying only on the manually defined signatures, making them more adaptable to modern cyber threats.

Datasets such as the chosen for this study the CICIDS2018 provide realistic network traffic simulations that include both benign behaviour and a multiple cyberattacks such as DoS, DDoS, brute-force, and botnet traffic (Han et al., 2024; Canadian Institute for Cybersecurity 2018). Despite being introduced in 2018, this dataset remains widely used in intrusion detection research due to its detailed flow-based features, realistic attack simulations, and labelled multi-class structure, which continue to reflect modern network traffic characteristics and allow fair benchmarking of machine learning models. However, these datasets also introduce challenges such as high dimensionality, noise and most importantly severe class imbalance, where benign traffic significantly outweighs malicious samples.

1.2 Problem Definition

One of the biggest challenges in machine learning that is based intrusion detection is that datasets are highly imbalanced generally, meaning that normal (benign) network traffic significantly outnumbers malicious attack samples (Widodo, Setiawan and Indraswari, 2024). In real-world and simulated network environments, normal traffic dominates while attack traffic represents only a small fraction of overall data. When network traffic is captured and converted it into a structured datasets for analysis, this imbalance is directly reflected in the training data used by machine learning models (Sharafaldin, Lashkari and Ghorbani, 2018). That imbalance causes machine learning models to become biased toward the majority classes in the datasets because of that often resulting in high overall accuracy but poor detection of minority attack classes. This issue is also reflected in the CICIDS2018 dataset, where benign traffic significantly outweighs attack classes (Sharafaldin, Lashkari and Ghorbani, 2018).

In the context of intrusion detection, this is particularly problematic because misclassification of rare attack types (false negatives) can lead to undetected security breaches. For that reason, improving minority class detection is more important than achieving high accuracy by itself.

This research highlight and focuses on how to handle the challenge of class imbalance in network intrusion detection using the CICIDS2018 dataset and investigates whether resampling techniques can improve the performance of machine learning models in identifying diverse attack types.

1.3 Aim

The aim of this research is to develop and evaluate a machine learning-based intrusion detection system capable of accurately classifying network traffic into benign and malicious categories while addressing the issue of class imbalance. In this context, machine learning provides a broad framework for intrusion detection, while ensemble learning methods are considered due to their improved robustness and performance on imbalanced datasets. Among these, Random Forest is selected as the primary classifier based on its strong generalisation ability and effectiveness in handling high-dimensional network traffic data.

1.4 Objectives

The main objectives of this study are:

- To collect and prepare labelled network traffic data from the selected CICIDS2018 dataset and the merge of the selected subsets
- To perform data preprocessing such as cleaning, missing value handling missing values and to remove duplicated data
- To identify and select for classification the relevant network flow features
- To develop a Random Forest-based classifier for intrusion detection
- To apply and evaluate class imbalance handling techniques, including SMOTE and Borderline-SMOTE
- To compare the model different configurations performance using standard evaluation metrics like accuracy and macro F1-score

1.5 Hypothesis

It is hypothesised that machine learning models, specifically Random Forest classifiers are able to effectively detect and classify network intrusion traffic, and also that their performance can be significantly improved through the application of various class imbalance handling techniques such as SMOTE and Borderline-SMOTE (Amachaghi et al., 2025; Saputra et al., 2025).

1.6 Research Questions

This study focuses on to answer the following research questions:

- Can a Random Forest-based machine learning model accurately distinguish between benign and malicious network traffic?
- Does addressing class imbalance improve the detection of minority attack classes?
- Which resampling technique, SMOTE or Borderline-SMOTE, provides better overall performance for intrusion detection?

1.7 Scope of the Study

This research focuses on supervised machine learning-based intrusion detection using structured network flow data derived from CICIDS2018 subsets. The study is limited only to flow-based features and does not incorporate with deep packet inspection or any real-time packet analysis. The focus is on the offline classification performance using labelled datasets rather than on a live deployment.

Chapter 2: Literature Review

2.1 Introduction

Intrusion Detection Systems or IDS play a critical role in modern cybersecurity due to the fact it can monitor network traffic and identify malicious or suspicious activity. As cyber threats complexity and frequency evolved greatly in the past few years, traditional security mechanisms have become less effective at detecting advanced or previously unseen attack patterns, mainly because they rely on predefined signatures or rules that cannot generalise to novel or evolving attack behaviours. The result for that an increased interest in the machine learning-based intrusion detection systems, what can learn behavioural patterns directly from data (Diana, Dini and Paolini, 2025; Sithole and Du Toit, 2022, 368; Hnamte and Hussain, 2023).

This chapter reviews the important and key concepts what is related to intrusion detection systems, machine learning applications in cybersecurity, dataset challenges such as class imbalance, and resampling techniques including Synthetic Minority Oversampling Technique (SMOTE) and Borderline-SMOTE. The aim of it is to establish the theoretical foundation for this study and identify relevant gaps addressed in this research.

2.2 Intrusion Detection Systems

Intrusion Detection Systems can generally be categorised into two main types: signature-based and anomaly-based systems (Ali et al., 2025; MahdaviFar and Ghorbani, 2019).

Signature based IDS are predefined with rules or known attack signatures to be able to detect malicious activity on the network where they are deployed. These types of IDS are highly effective for identifying previously known threats but limited to detect novel or modified attacks that do not match existing patterns.

On the other hand, anomaly-based IDS are aimed to identify deviations from normal network behaviour. These systems much more adaptable to unknown attacks but often suffer from the higher false positive rates due to the difficulty in accurately differentiate normal traffic behaviour from malicious ones.

Due to these limitations, machine learning-based approaches have been increasingly adopted as they allow models to learn complex patterns directly from network data without requiring explicit rule definitions.

2.3 Machine Learning in intrusion detection systems (IDS)

Machine learning has become a widely used approach in the past few years in intrusion detection due to its ability to classify complex data. Many of the supervised learning algorithms have been applied to network security tasks such as Decision Trees, Support Vector Machines, Neural Networks, and ensemble methods such as Random Forest (Tsai et al., 2009; Genuario et al., 2024).

Random Forest is highly effective in intrusion detection applications due to several advantages it has:

- It performs really well when working with structured, tabular data such as the records of the network flow where patterns and relationships between the variables can be clearly defined and analysed
- It helps reduce the risk of overfitting carrying it out by combining the predictions of multiple decision trees which improves the overall stability and makes the model more reliable for unseen data
- It generally provides strong and reliable classification performance across a wide range of datasets and those subsets, making it a robust choice for different types of problems

Previous study has demonstrated that Random Forest classifiers achieve strong and reliable performance in intrusion detection systems, particularly when combined with appropriate preprocessing and feature selection techniques (Saputra et al., 2025).

2.4 Network Traffic Datasets

The effectiveness of machine learning-based IDS models depends heavily on the quality of the dataset used for the training and evaluation. Commonly used datasets in cybersecurity research are the CICIDS2017 and the CICIDS2018 (our project), which simulate realistic network environments and contain multiple types of attack labels (Canadian Institute for Cybersecurity 2017; Canadian Institute for Cybersecurity 2018).

The CICIDS2018 dataset includes various forms of network attacks such as brute-force attacks, denial-of-service (DoS), distributed denial-of-service (DDoS), and botnet activity just to name some of them. Each record represents a network flow and contains statistic features describing communication between source and destination endpoints. Compared to older datasets such as KDD Cup 99 and NSL-KDD, CICIDS2018 better reflects contemporary network behaviour and attack patterns, making it more suitable for evaluating modern intrusion detection systems.

However, a major limitation of these datasets is the presence of severe class imbalance, where benign traffic significantly outweighs attack traffic. Thanks to this imbalance it raises challenges for machine learning models, often leading to benign predictions toward to the majority classes.

2.5 Class Imbalance Problem

Class imbalance is a well-known issue in machine learning-based intrusion detection systems. In real-world network environments, normal traffic occurs far more frequently than malicious activity. As a result, models trained on imbalanced data tend to favour majority classes while underperforming on minority attack classes.

This leads to several issues:

- High overall accuracy but the minority class are poorly detected
- Increased false negatives for the rare attack types
- Reduced practical effectiveness in cybersecurity applications

In intrusion detection systems or IDS, false negatives should be minimised as they represent malicious activities that are incorrectly classified as benign. Such errors can have severe real-world consequences, including undetected intrusions, data breaches, and system compromise.

As a result for that, reducing false negatives is a key objective in intrusion detection research, particularly in imbalanced datasets where attack classes are underrepresented.

2.6 SMOTE and Borderline-SMOTE

To fix the class imbalance problem one of the most widely used methods was applied the Synthetic Minority Oversampling Technique SMOTE (Chawla et al., 2002; Thejas et al., 2022). This technique basically generates synthetic samples for the minority classes doing this by estimating between existing data points. SMOTE helps balance class distributions and improves the model's ability to learn minority class patterns without simply duplicating existing samples.

Borderline-SMOTE is an extension of the original SMOTE model that focuses on generating synthetic samples near the class boundaries, this is where misclassification is more likely to happen. By concentrating on the borderline regions between the classes, it basically aims to improve the classifier's ability to differentiate between overlapping or similar classes.

Both techniques are widely used in cybersecurity research and are particularly relevant for intrusion detection datasets with highly imbalanced class distributions. These techniques were selected over alternatives such as under sampling and cost-sensitive learning because they preserve the original dataset distribution while improving minority class representation, which is important in intrusion detection datasets (Chawla et al., 2002; Han, Wang and Mao, 2005). Under sampling may lead to loss of useful information, while cost-sensitive methods require careful tuning of misclassification costs.

2.7 Related Work

Previous studies have demonstrated and highlighted the effectiveness of machine learning models in intrusion detection systems, mainly ensemble methods such as Random Forest (Tsai et al., 2009).

Studies have also shown that class imbalance significantly affects model performance in cybersecurity applications and they have reported that many models achieve high overall accuracy but significantly lower detection performance for minority attack classes when

imbalance is not properly addressed (Mittal, Lallie and Titis 2025; Chawla et al., 2002; Tsai et al., 2009).

To mitigate that issue, resampling techniques just like SMOTE (Synthetic Minority Oversampling Technique) have been widely adopted in intrusion detection research (Chawla et al., 2002). These methods improve minority class detection and enhance overall model robustness.

2.8 Research Gap

Despite that significant advancements happened in machine learning-based intrusion detection systems, several gaps still remain in the literature:

Many studies based on intrusion detection historically prioritise overall accuracy as the primary or number one evaluation metric, despite evidence that accuracy can be misleading in imbalanced datasets (Tsai et al., 2009; Chawla et al., 2002; Saputra et al., 2025).

Also have to mention that while SMOTE has been widely adopted for handling class imbalance (Chawla et al., 2002), some studies demonstrate and highlight direct comparative evaluations with Borderline-SMOTE in modern intrusion detection pipelines (Wu et al., 2022).

Furthermore, most comparative studies are conducted on benchmark datasets such as NSL-KDD or CICIDS2017, while fewer studies explore CICIDS2018 subsets with flow-based feature selection in multi-class classification settings features (Sharafaldin, Lashkari and Ghorbani, 2018).

This research addresses these gaps by providing a comparative evaluation of SMOTE and Borderline-SMOTE using a Random Forest classifier (Han, Wang and Mao, 2005), with a focus on macro-averaged F1-score to ensure balanced performance across all classes.

2.9 Summary

This chapter reviewed key concepts that is related to intrusion detection systems, machine learning applications in cybersecurity, dataset challenges and class imbalance handling techniques. The literature highlights the importance of addressing imbalanced data distributions and supports the use of ensemble learning methods such as Random Forest for network traffic classification.

The identified research gap provides the foundation for the methodology and experimental design presented in the following chapter.

Chapter 3: Methodology

3.1 Overview

The following chapter describe the methodology what was used to develop and evaluate a machine learning-based intrusion detection system. The process follows a structured pipeline consisting of dataset selection, data preprocessing, feature handling, model training, class imbalance handling, and performance evaluation.

The objective of this study is to build a supervised learning model capable of distinguishing between benign and malicious network traffic while addressing the significant class imbalance present in the dataset.

This study follows the CRISP-DM framework to structure the machine learning workflow. CRISP-DM is widely used in industry and research as it give a systematic and iterative approach to data mining projects.

The CRISP-DM framework consists of six phases: business understanding, data understanding, data preparation, modelling, evaluation, and deployment. In this research, these phases are applied as follows:

- **Business Understanding:**

The objective of this phase is to develop a machine learning-based intrusion detection system capable of improving detection of malicious network traffic and reducing false negatives in imbalanced datasets.

- **Data Understanding:**

To collect and prepare labelled network traffic data from the selected CICIDS2018 dataset and merged subsets and understand class distribution and attack types.

- **Data Preparation:**

To carry out preprocessing just like data cleaning, handling missing values, removing duplicates, and selecting relevant network flow features for classification.

- **Modelling:**
To develop a Random Forest-based classifier for intrusion detection to distinguish between benign and malicious traffic.
- **Evaluation:**
To apply and evaluate the class imbalance techniques were used (SMOTE and Borderline-SMOTE) and compare model performance using accuracy and macro F1-score.
- **Deployment (Future Work):**
To consider real-time deployment of the intrusion detection system in future work.

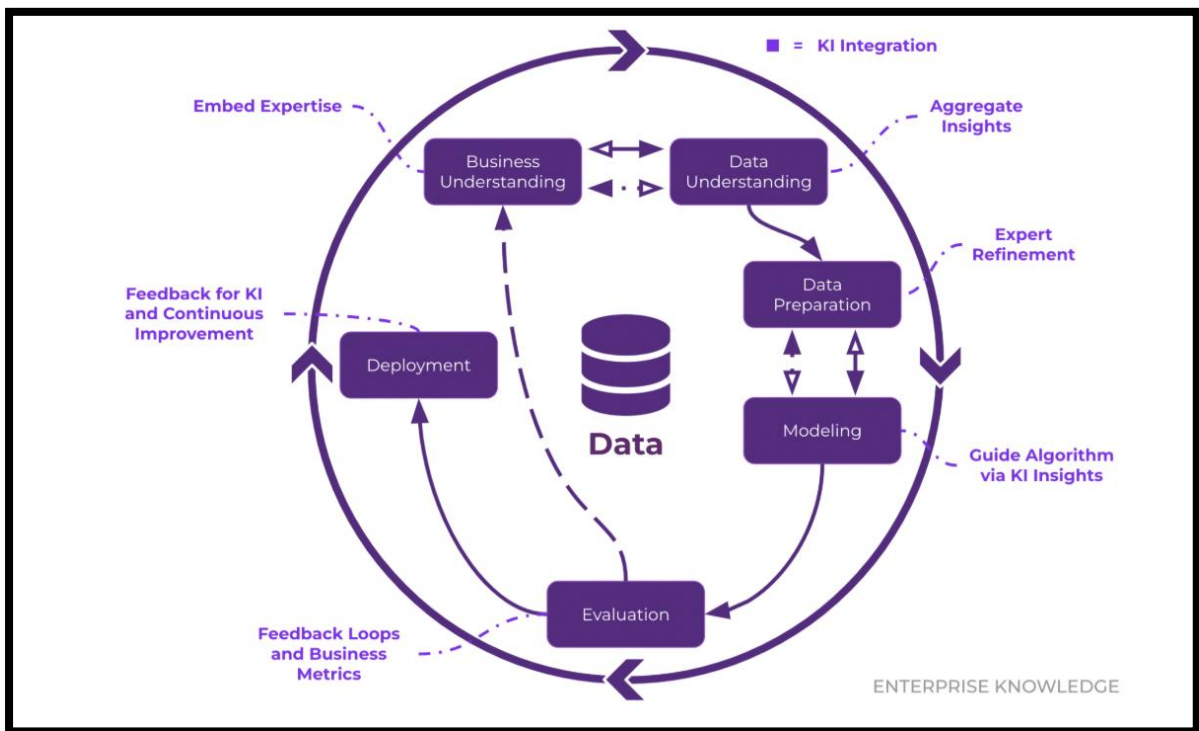


Figure 1. CRISP-DM framework phases

Source: <https://enterprise-knowledge.com/understanding-the-role-of-knowledge-intelligence-in-the-crisp-dm-framework-a-guide-for-data-science-projects/>

3.2 Dataset and Understanding

This study uses selected subsets of the CICIDS2018 dataset, which is widely used in network intrusion detection research due to its realistic simulation of enterprise network traffic and diverse attack scenarios.

Each record in the dataset represents a network communication flow between devices on a network. The features describe characteristics of this communication, such as the number of packets sent, the amount of transferred data, the duration of the connection, and the behaviour of TCP control flags. To put it simple these features help describe how devices communicate over the network and allow machine learning models to identify patterns associated with normal or malicious activity.

The dataset of the subsets was used in this research includes the following attack types:

- Benign traffic
- Botnet activity
- DoS Hulk attacks
- DDoS LOIC-HTTP attacks
- DDoS HOIC attacks
- FTP-Brute Force attacks
- SSH-Bruteforce attacks
- DoS SlowHTTPTest attacks
- DDoS LOIC-UDP attacks

The selected attack types represent common forms of malicious network behaviour, including flooding attacks (DoS/DDoS), unauthorised access attempts (FTP and SSH brute force), and botnet activity. These categories were chosen because they provide a variety of attack patterns and allow the model to be evaluated across different types of network intrusions while maintaining a manageable and balanced subset of the CICIDS2018 dataset.

After merging the dataset contained over 11 million samples, making it a large-scale classification problem with severe class imbalance. An exploratory data analysis or EDA was conducted to better understand the dataset before preprocessing and model training. This included examining class distributions, identifying missing values and duplicate records, analysing feature data types, and reviewing basic statistical properties of the selected network flow features. The analysis found a severe class imbalance, how the benign traffic significantly outnumbered several attack categories.

Datasets merged successfully. Merged shape: (12143848, 84)

	Dst Port	Protocol	Timestamp	Flow Duration	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Fwd Pkt Len Max	Fwd Pkt Len Min	Fwd Pkt Len Mean	Fwd Pkt Len Std	Bwd Pkt Len Max	Bwd Pkt Len Min	Bwd Pkt Len Mean	Bwd Pkt Len Std	Flow Byts/s	Flow Pkts/s	Flow IAT Mean
0	0	0	14/02/2018 08:31:01	112641719	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56320859.500
1	0	0	14/02/2018 08:33:50	112641466	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56320733.000
2	0	0	14/02/2018 08:36:39	112638623	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56319311.500
3	22	6	14/02/2018 08:40:13	6453966	15	10	1239	2273	744	0	82.600	196.741	976	0	227.300	371.678	544.162	3.874	268915.250
4	22	6	14/02/2018 08:40:23	8804066	14	11	1143	2209	744	0	81.643	203.746	976	0	200.818	362.250	380.733	2.840	366836.083

Merged dataset saved as 'merged_dataset.csv'.
Individual DataFrames cleared from memory.

Figure 2. Merged subsets dataset more than 11 million samples

Columns with missing values:

Missing Count	
Flow Byts/s	41602
Flow ID	4194300
Src IP	4194300
Src Port	4194300
Dst IP	4194300

Found 396232 duplicate rows.

Figure 3. Columns with missing values found

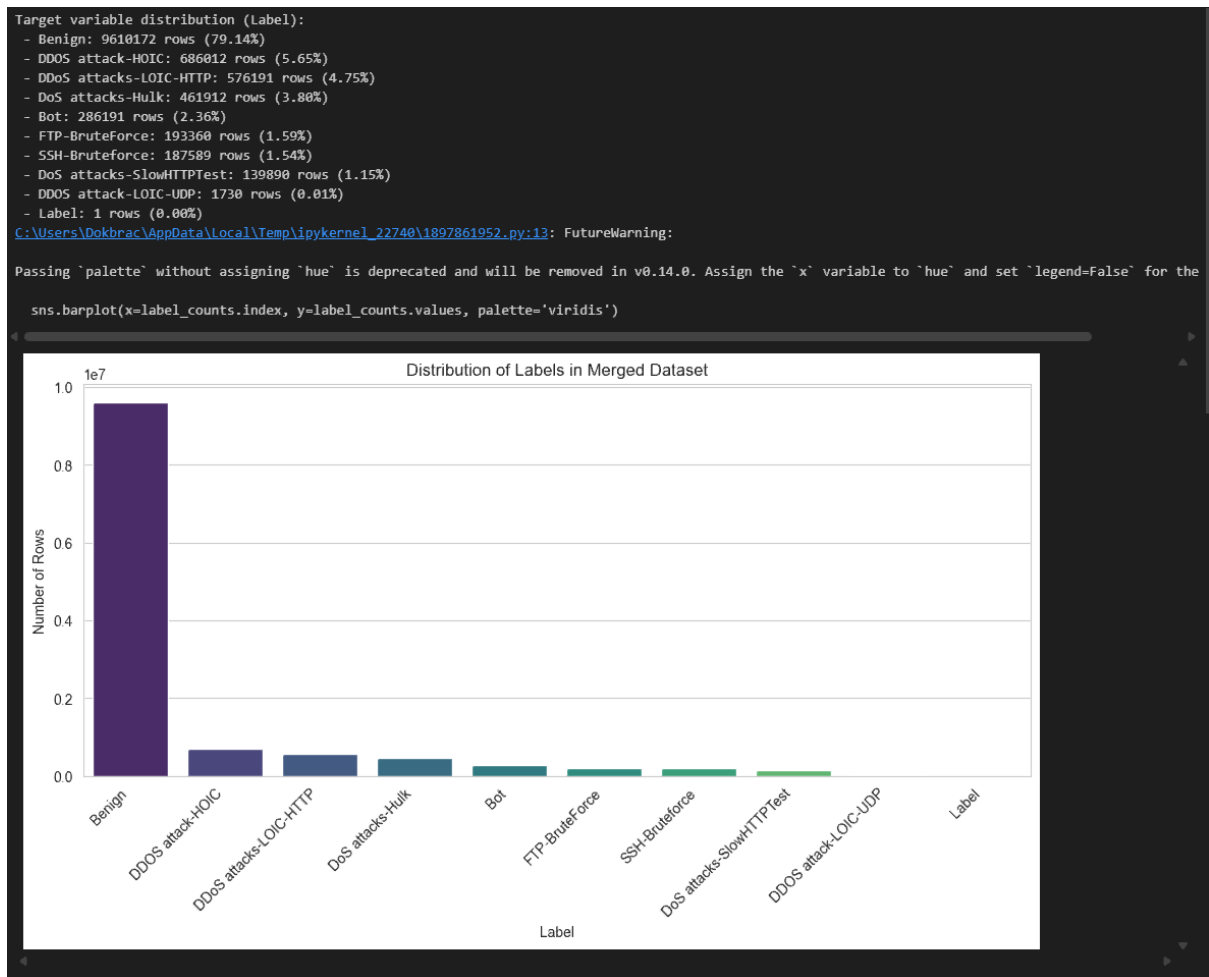


Figure 4. Target variable distribution

3.3 Data Preprocessing

Data understanding phase conducted in the beginning in order to explore the structure and characteristics of the CICIDS2018 dataset before any preprocessing or model training. It is included of examining the distribution of class labels, particularly the percentage of benign versus attack traffic to know the severity of class imbalance in the dataset.

The analysis found that benign traffic dominates the dataset in a big part, while on the other hand the attack categories such as botnet activity, brute-force attacks, and various DDoS/DoS variants are underrepresented. This imbalance reflects real-world network environments where malicious activity occurs less frequently than normal traffic.

In addition, the dataset was inspected at feature level to understand the nature of the input variables. The features are flow-based and describe statistical properties of network communication, including packet counts, flow duration, and TCP flag behaviours. This

understanding informed subsequent preprocessing steps and justified the use of resampling techniques such as SMOTE and Borderline-SMOTE to address class imbalance.

A couple of preprocessing steps were taken in order to ensure the quality and consistency of data:

Missing Value Handling

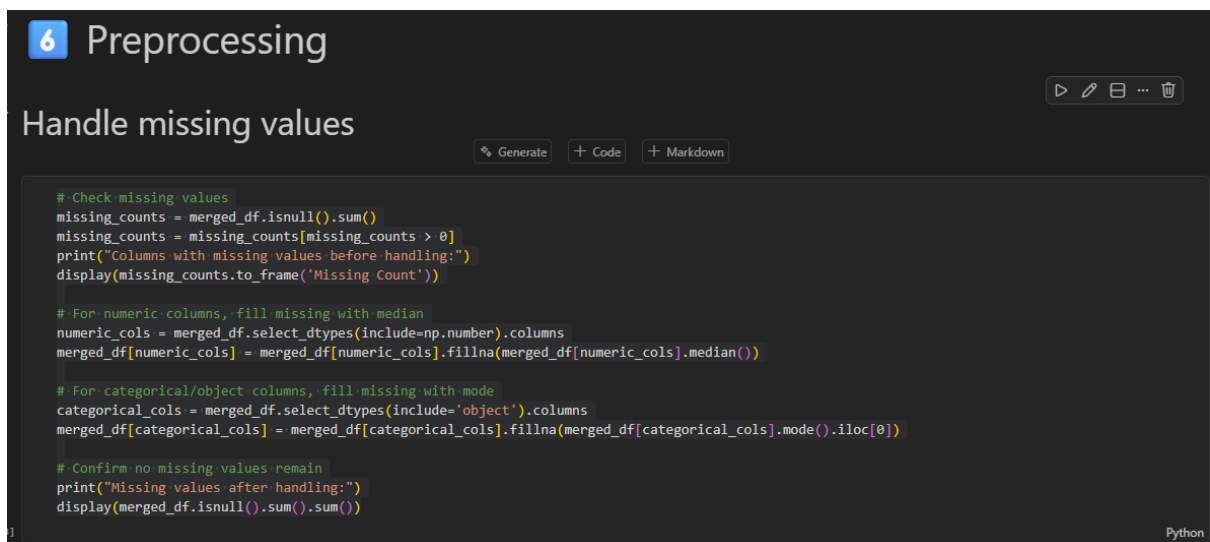
- Numerical features were filled out using median value
- Categorical features were filled out using the most frequent (mode) value

Duplicate Removal

- Approximately 396,232 duplicate rows were identified and removed. These were exact duplicate records (identical across all feature columns), and their removal was performed to prevent redundancy and potential bias in model training

Data Type Conversion

- All features were converted into an appropriate numeric format to ensure compatibility with machine learning algorithms require numerical input for training and prediction.



```
# Check missing values
missing_counts = merged_df.isnull().sum()
missing_counts = missing_counts[missing_counts > 0]
print("Columns with missing values before handling:")
display(missing_counts.to_frame("Missing Count"))

# For numeric columns, fill missing with median
numeric_cols = merged_df.select_dtypes(include=np.number).columns
merged_df[numeric_cols] = merged_df[numeric_cols].fillna(merged_df[numeric_cols].median())

# For categorical/object columns, fill missing with mode
categorical_cols = merged_df.select_dtypes(include='object').columns
merged_df[categorical_cols] = merged_df[categorical_cols].fillna(merged_df[categorical_cols].mode().iloc[0])

# Confirm no missing values remain
print("Missing values after handling:")
display(merged_df.isnull().sum().sum())
```

Figure 5. Code to handle missing values

Columns with missing values before handling:

	Missing Count
Flow Byts/s	41602
Flow ID	4194299
Src IP	4194299
Src Port	4194299
Dst IP	4194299

C:\Users\Dokbrac\AppData\Local\Temp\ipykernel_22740\4053510586.py:12: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes. See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with categorical_cols = merged_df.select_dtypes(include='object').columns

Missing values after handling:

Figure 6. Missing values before and after handling it

6 Preprocessing

Handle missing values

1 cell hidden ...

Encode categorical variables

```
# Initialize LabelEncoder
le = LabelEncoder()

# Fit and transform the Label column
merged_df['Label_encoded'] = le.fit_transform(merged_df['Label'])

# Display the mapping
label_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print("Label encoding completed. Mapping of attacks to integers:")
for label, code in label_mapping.items():
    print(f"{label} --> {code}")

# Preview encoded column
display(merged_df[['Label', 'Label_encoded']].head())

# Remove extremely rare classes BEFORE split

class_counts = merged_df['Label_encoded'].value_counts()

threshold = 50

rare_classes = class_counts[class_counts < threshold].index

print("Removing rare classes:", rare_classes)

merged_df = merged_df[~merged_df['Label_encoded'].isin(rare_classes)]

print("Shape after removing rare classes:", merged_df.shape)

# Save Encoder
joblib.dump(le, "label_encoder.pkl")
print("Label encoder saved")

print("New class distribution:")
print(merged_df['Label_encoded'].value_counts())
```

[14] Python

Figure 7. Code for numerical format

```

Label encoding completed. Mapping of attacks to integers:
Benign --> 0
Bot --> 1
DDoS attack-HOIC --> 2
DDoS attack-LOIC-UDP --> 3
DDoS attacks-LOIC-HTTP --> 4
DoS attacks-Hulk --> 5
DoS attacks-SlowHTTPTest --> 6
FTP-BruteForce --> 7
SSH-BruteForce --> 8

  Label  Label_encoded
0  Benign           0
1  Benign           0
2  Benign           0
3  Benign           0
4  Benign           0

Removing rare classes: Index([], dtype='int64', name='Label_encoded')
Shape after removing rare classes: (12143047, 85)
Label encoder saved
New class distribution:
Label_encoded
0      9610172
2      686012
4      576191
5      461912
1      286191
7      193360
8      187589
6      139890
3         1730
Name: count, dtype: int64

```

Figure 8. Output of the numeric mapping

6 Preprocessing

- Handle missing values
 - 1 cell hidden ...
- Encode categorical variables
 - 1 cell hidden ...
- Select relevant features

Generate + Code + Markdown

```

# Check for duplicate rows and remove them
duplicate_count = merged_df.duplicated().sum()
print(f"Found {duplicate_count} duplicate rows.")
merged_df = merged_df.drop_duplicates()
print(f"Dataset shape after removing duplicates: {merged_df.shape}")

# Check for constant or zero-variance columns
constant_cols = [col for col in merged_df.columns if merged_df[col].nunique() <= 1]
print(f"Constant/Zero-variance columns: {constant_cols}")
merged_df = merged_df.drop(columns=constant_cols, errors='ignore')
print(f"Dataset shape after dropping constant columns: {merged_df.shape}")

# Select relevant features
selected_features = [
    'Tot Fwd Pkts', 'Tot Bwd Pkts',
    'TotLen Fwd Pkts', 'TotLen Bwd Pkts',
    'Flow Duration', 'SYN Flag Cnt', 'FIN Flag Cnt',
    'RST Flag Cnt', 'PSH Flag Cnt', 'ACK Flag Cnt', 'URG Flag Cnt'
]

# Ensure selected features exist in the dataset
selected_features = [f for f in selected_features if f in merged_df.columns]
print(f"Selected features: ({len(selected_features)} columns): {selected_features}")

# Separate features (X) and target (y)
X = merged_df[selected_features]
y = merged_df['Label_encoded']

# Save the cleaned and selected-feature dataset
cleaned_dataset = pd.concat([X, y], axis=1)
cleaned_dataset.to_csv("cleaned_selected_features_dataset.csv", index=False)
print("Cleaned dataset saved as 'cleaned_selected_features_dataset.csv'")

# Preview
display(cleaned_dataset.head())

```

Figure 9. Code to set the relevant features and remove the duplicated rows

```

Found 396232 duplicate rows.
Dataset shape after removing duplicates: (11746815, 85)
Constant/Zero-variance columns: ['Bwd PSH Flags', 'Fwd URG Flags', 'Bwd URG Flags', 'CWE Flag Count', 'Fwd Byts/b Avg', 'Fwd Pkts/b Avg', 'Fwd Blk Rate']
Dataset shape after dropping constant columns: (11746815, 75)
Selected features (11 columns): ['Tot Fwd Pkts', 'Tot Bwd Pkts', 'TotLen Fwd Pkts', 'TotLen Bwd Pkts', 'Flow Duration', 'SYN Flag Cnt', 'FIN Flag Cnt', 'RST Flag Cnt', 'PSH Flag Cnt', 'ACK Flag Cnt', 'URG Flag Cnt', 'Label_encoded']
Cleaned dataset saved as 'cleaned_selected_features_dataset.csv'

```

	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Flow Duration	SYN Flag Cnt	FIN Flag Cnt	RST Flag Cnt	PSH Flag Cnt	ACK Flag Cnt	URG Flag Cnt	Label_encoded
0	3	0	0	0	112641719	0	0	0	0	0	0	0
1	3	0	0	0	112641466	0	0	0	0	0	0	0
2	3	0	0	0	112638623	0	0	0	0	0	0	0
3	15	10	1239	2273	6453966	0	0	0	1	0	0	0
4	14	11	1143	2209	8804066	0	0	0	1	0	0	0

Figure 10. Output after the feature selection and duplicated rows removal

After preprocessing, the dataset was reduced in dimensionality while preserving meaningful information for classification. As visible on the Figures the preprocessing pipeline successfully handled the missing values problem, removed the duplicate records, and converted all features into a machine-learning compatible format. The dataset is now clean, consistent, and free from missing or redundant data, making it suitable for training the classification configurations.

3.4 Feature Selection

The following 11 features were selected:

- Total forward packets
- Total backward packets
- Total length of forward packets
- Total length of backward packets
- Flow duration
- SYN flag count
- FIN flag count
- RST flag count
- PSH flag count
- ACK flag count
- URG flag count

The feature selection happened manually based on domain knowledge of network traffic behaviour and its relevance to intrusion detection systems (IDS). The selected features were chosen because they are known to effectively capture behavioural differences between benign and malicious traffic patterns in flow-based analysis (Hindy et al., 2021).

In particular, packet-level metrics, flow duration, and TCP flag statistics was the priority because they are strongly associated with common attack behaviours in the CICIDS2018 dataset. For example, denial-of-service (DoS) and distributed denial-of-service (DDoS) attacks typically generate unusually high packet volumes and flow lengths, while brute-force and scanning-based attacks often exhibit distinctive patterns in connection frequency and control flag usage (e.g., SYN and ACK activity).

3.5 Model Selection

Random Forest classifier was used as the main and only machine learning model for this study. It is an ensemble learning method that create multiple decision trees at the same time and collects and merge the predictions to improve the performance of the classification.

It was chosen due to the following advantages:

- Works very well with structured, tabular data.
- Without slowing down too much it can handle large datasets very well.
- It generalizes well to new data because it is resistant to overfitting.
- Between the features it is able to capture complex non-linear relationships.
- Support parallel processing (e.g., `n_jobs = -1`), making it faster and more scalable.

Beside Random Forest other common machine learning models were under the consideration to evaluate which is the best for this study namely Logistic Regression, Support Vector Machines (SVM), and k-Nearest Neighbours (KNN). However, these were not selected for the final model due to their limitations in this context:

- Logistic Regression: limited in capturing non-linear relationships present in network traffic behaviour.
- SVM: computationally expensive on large datasets such as CICIDS2018 (over 11 million samples after merging).

- KNN: high memory usage and slow prediction time on large-scale datasets.

In contrast, Random Forest provides a balance between performance, scalability, and interpretability, making it suitable for intrusion detection tasks involving large, imbalanced network traffic datasets.

The model was trained firstly after the selected and merged sub datasets using a baseline configuration and later another 4 configuration was trained and were evaluated under different class imbalance handling strategies.

3.6 Class Imbalance Handling

The dataset showed a noticeable class imbalance as was expected, where benign traffic dominated most subsets. Just an example for it namely in the 02-14-2018 dataset, benign traffic accounted for 63.67% of records, while attack classes such as FTP-BruteForce (18.44%) and SSH-Bruteforce (17.89%) were significantly smaller. The imbalance was even more severe in others, such as 02-15-2018, where benign traffic reached 94.99%, leaving only 5.01% for attack classes combined. Similar patterns were observed across multiple datasets, with benign traffic consistently exceeding 70–99% in most cases. This imbalance can bias machine learning models toward to the majority class predictions and reduce the detection performance of minority attack categories, so the model won't be able to recognise those attacks.

```

02-14-2018.csv:
  Label column present = True
  Unique values count = 3
  Unique value names = <StringArray>
['Benign', 'FTP-BruteForce', 'SSH-Bruteforce']
Length: 3, dtype: str
  Value percentages:
    Benign: 63.67%
    FTP-BruteForce: 18.44%
    SSH-Bruteforce: 17.89%
-----

02-15-2018.csv:
  Label column present = True
  Unique values count = 3
  Unique value names = <StringArray>
['Benign', 'DoS attacks-GoldenEye', 'DoS attacks-Slowloris']
Length: 3, dtype: str
  Value percentages:
    Benign: 94.99%
    DoS attacks-GoldenEye: 3.96%
    DoS attacks-Slowloris: 1.05%
-----

```

(a)

```

-----
02-16-2018.csv:
  Label column present = True
  Unique values count = 4
  Unique value names = <StringArray>
['Benign', 'DoS attacks-SlowHTTPTest', 'DoS attacks-Hulk', 'Label']
Length: 4, dtype: str
  Value percentages:
    DoS attacks-Hulk: 44.05%
    Benign: 42.61%
    DoS attacks-SlowHTTPTest: 13.34%
    Label: 0.00%
-----

02-20-2018.csv:
  Label column present = True
  Unique values count = 2
  Unique value names = <StringArray>
['Benign', 'DDoS attacks-LOIC-HTTP']
Length: 2, dtype: str
  Value percentages:
    Benign: 92.75%
    DDoS attacks-LOIC-HTTP: 7.25%
-----

```

(b)

```

-----
02-21-2018.csv:
  Label column present = True
  Unique values count = 3
  Unique value names = <StringArray>
['Benign', 'DDOS attack-LOIC-UDP', 'DDOS attack-HOIC']
Length: 3, dtype: str
  Value percentages:
    DDOS attack-HOIC: 65.42%
    Benign: 34.41%
    DDOS attack-LOIC-UDP: 0.16%
-----

02-22-2018.csv:
  Label column present = True
  Unique values count = 4
  Unique value names = <StringArray>
['Benign', 'Brute Force -Web', 'Brute Force -XSS', 'SQL Injection']
Length: 4, dtype: str
  Value percentages:
    Benign: 99.97%
    Brute Force -Web: 0.02%
    Brute Force -XSS: 0.01%
    SQL Injection: 0.00%
-----

```

(c)

```

-----
02-23-2018.csv:
  Label column present = True
  Unique values count = 4
  Unique value names = <StringArray>
['Benign', 'Brute Force -Web', 'Brute Force -XSS', 'SQL Injection']
Length: 4, dtype: str
  Value percentages:
    Benign: 99.95%
    Brute Force -Web: 0.03%
    Brute Force -XSS: 0.01%
    SQL Injection: 0.01%
-----

02-28-2018.csv:
  Label column present = True
  Unique values count = 3
  Unique value names = <StringArray>
['Benign', 'Label', 'Infiltration']
Length: 3, dtype: str
  Value percentages:
    Benign: 88.76%
    Infiltration: 11.23%
    Label: 0.01%
-----

```

(d)

```

-----
03-01-2018.csv:
  Label column present = True
  Unique values count = 3
  Unique value names = <StringArray>
['Benign', 'Label', 'Infiltration']
Length: 3, dtype: str
  Value percentages:
    Benign: 71.89%
    Infiltration: 28.11%
    Label: 0.01%
-----
03-02-2018.csv:
  Label column present = True
  Unique values count = 2
  Unique value names = <StringArray>
['Benign', 'Bot']
Length: 2, dtype: str
  Value percentages:
    Benign: 72.71%
    Bot: 27.29%
-----
Label column inspection completed with percentages.

```

(e)

Figure 11. Output of the imbalanced classes in the original dataset: (a), (b), (c), (d) & (e)

To handle that issue two oversampling techniques from the Imbalanced-learn library were used: SMOTE and Borderline-SMOTE. Both methods were used only on the training dataset after the train-test split to avoid data leakage while the test set remained unchanged to ensure an unbiased evaluation.

SMOTE (Synthetic Minority Oversampling Technique) generates synthetic samples for minority classes by combining between existing data points. This helps to improve class distribution and allows the model to learn more representative decision boundaries for underrepresented classes.

In this study SMOTE was used to balance minority classes based on class distribution inside the training set. The objective was to improve overall class representation without excessively inflating synthetic data.

Borderline-SMOTE is an extension of SMOTE that focuses on generating synthetic samples near decision boundaries between classes. This approach focusing on harder-to-classify instances where misclassification is more likely to happen.

Targeting these borderline regions the Borderline-SMOTE aim is to improve the model ability to differentiate between the overlapping classes.

Both techniques were evaluated independently and in combination with Random Forest hyperparameter tuning to review their impact performance on classification with the identical evaluation conditions.

3.7 Train-Test Split Strategy

The dataset was split into a training and a testing set and a stratified sampling approach were used to keep the original class distribution.

- Training set: 70%
- Testing set: 30%
- Stratification: applied on class labels

This ensured that both the training and the testing sets maintained similar class distributions made it available for fair evaluation of model performance.

After the splitting, additional rare-class filtering was applied to remove classes with extremely low representation in the training set to improve model stability.

3.8 Evaluation Metrics

The model performance was evaluated by using a set of standard classification metrics to calculate both overall accuracy and class-wise effectiveness focusing on because a severe class imbalance is present in the dataset.

TP (True Positive): attack correctly identified as attack

TN (True Negative): benign traffic correctly identified as benign

FP (False Positive): benign traffic incorrectly classified as attack

FN (False Negative): attack incorrectly classified as benign

Accuracy measures the proportion of correctly classified samples:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

However, due to the strong imbalance in the dataset (where benign traffic dominates up to 99% in some subsets), accuracy alone is not a reliable indicator of model performance.

To better evaluate classification quality across individual classes, the following metrics were used:

Precision evaluates the proportion of correctly predicted positive samples:

$$Precision = \frac{TP}{TP + FP}$$

Recall measures the ability of the model to correctly identify actual positive samples:

$$Recall = \frac{TP}{TP + FN}$$

F1-score provides a balance between precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

To account for multi-class imbalance, two aggregated versions of F1-score were used:

- **Macro F1-score:** calculate the F1-score independently for each class and assigns equal weight to all of the classes, making it particularly suitable for imbalanced datasets where minority classes are underrepresented.
- **Weighted F1-score:** calculate the F1-score for each class and averages them according to their support (class frequency), giving higher influence on majority classes such as Benign traffic.

From these metrics the **Macro F1-score** was selected as the most appropriate primary evaluation metric for this study. The reason for that is because it treats all classes equally

regardless of their frequency, so it is ensure that minority attack classes are not dominated by the benign traffic.

Per-class performance was further analysed using classification reports to evaluate how well the model detects individual attack types across the selected CICIDS2018 subsets.

3.9 Machine Learning Pipeline

The system follows a structured machine learning pipeline:

1. Setup Environment
 - Import libraries
 - Settings
2. Load & Inspect Datasets
 - List all dataset files
 - Inspect each dataset
 - Preview each dataset Label column
3. Merge Datasets
 - Load each selected dataset
 - Merge, Save, and Free Memory
4. Initial Exploration
 - Check missing values and duplicates
 - Explore target variable distribution
 - Update merged dataset
 - Visualize feature distributions
5. Early Baseline with Minimal Preprocessing
6. Preprocessing
 - Handle missing values

- Encode categorical variables
- Select relevant features

7. Train-Test Split

- Stratified Train-Test Split with Rare Class removal
- Label Check

8. Training

- Inspect class distribution in training data
- SMOTE(SAMPLING)
- SMOTE(SAMPLING) + HYPERPARAMETER TUNING
- BORDERLINE SMOTE(SAMPLING)
- BORDERLINE SMOTE(SAMPLING) + HYPERPARAMETER TUNING

9. Model Evaluation

- Load all results
- Create comparison table
- Visual comparison
- Automatically select best model
- Select + save final model
- Add Confusion Matrix
- Test real predictions
- Save All final artifacts

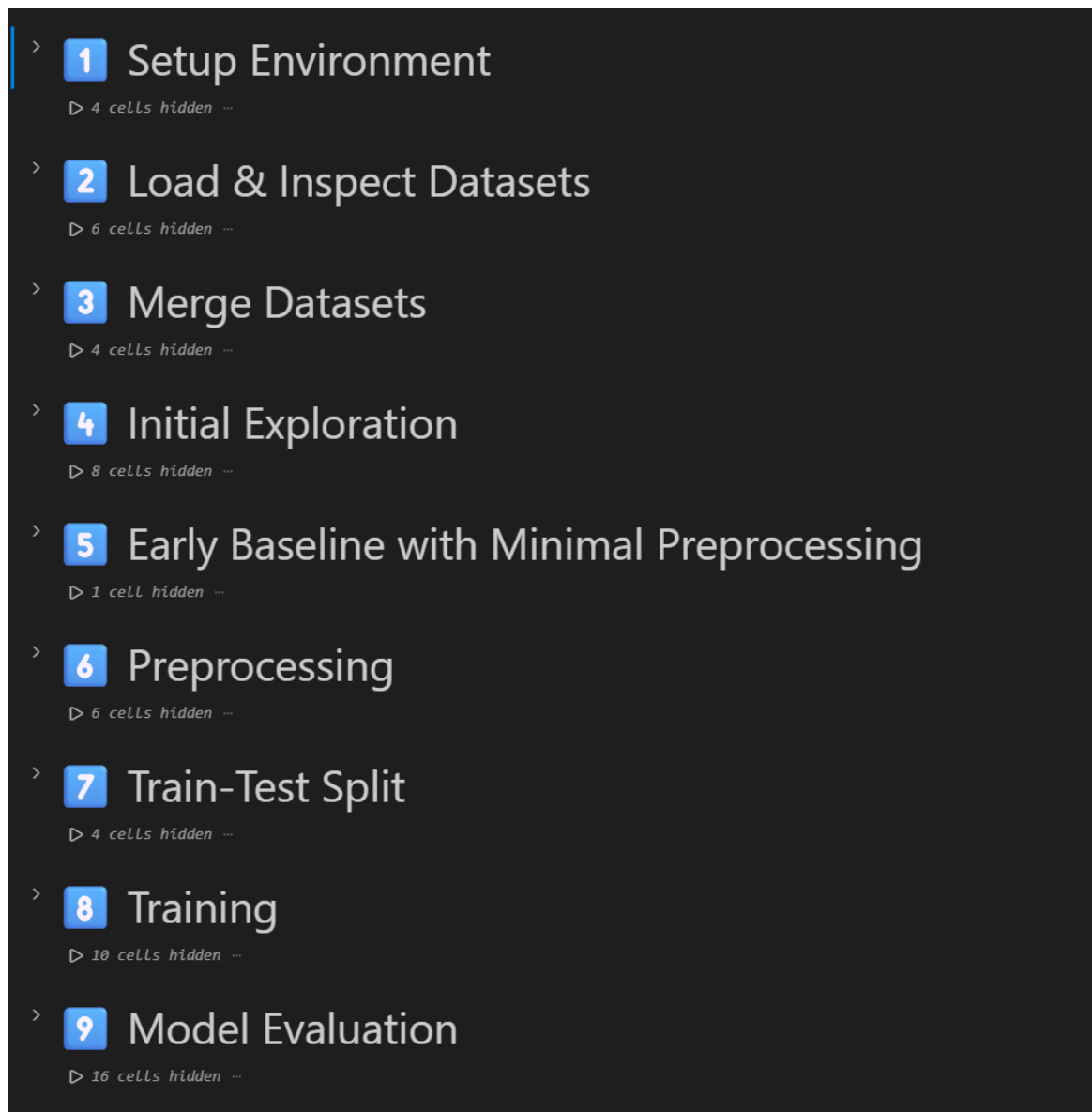


Figure 12. The Machine Learning Pipeline

This pipeline make sure that the model is trained in a controlled and reproducible environment for later evaluation while preventing data leakage.

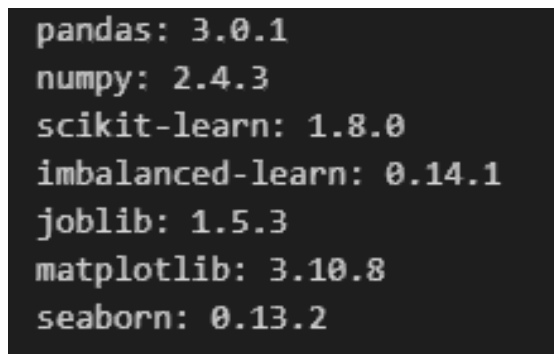
3.10 Computational Environment

The study and the experiments used Python and several scientific computing libraries. To ensure to be able the reproducibility the exact versions of all major libraries used in the study are reported below:

- pandas: 3.0.1

- numpy: 2.4.3
- scikit-learn: 1.8.0
- imbalanced-learn: 0.14.1
- joblib: 1.5.3
- matplotlib: 3.10.8
- seaborn: 0.13.2

The Random Forest classifier was trained using parallel processing (`n_jobs = -1`) to improve computational efficiency when handling the large-scale dataset.



```
pandas: 3.0.1
numpy: 2.4.3
scikit-learn: 1.8.0
imbalanced-learn: 0.14.1
joblib: 1.5.3
matplotlib: 3.10.8
seaborn: 0.13.2
```

Figure 13. Computational Environment

Chapter 4: Experiments and Results

4.1 Experimental Setup

The experimental test and the evaluation after it were carried out on a supervised machine learning pipeline what was implemented in Python. The system was developed by using Scikit-learn for the model training and evaluation phase and Imbalanced-learn for the techniques to handle imbalanced classes, including SMOTE and Borderline-SMOTE.

All experiments followed a consistent workflow to ensure comparability between the different configurations. The dataset was split using a stratified train-test split to preserve the original class distribution across both sets. Because of this it ensured that evaluation results reflected realistic network traffic proportions and avoided sampling bias.

The experiments were executed under a controlled environment using fixed random seeds to ensure reproducibility of results.

4.2 Baseline Configuration (Imbalanced Dataset)

An early baseline Random Forest classifier was trained on the original imbalanced dataset using minimal preprocessing and without any feature selection.

The results clearly demonstrate and highlight the effect of the class imbalance in present how it is affecting the model performance. Although the **overall accuracy is 0.3285**, this value is misleading because it is dominated by the majority class rather than reflecting balanced predictive performance across all classes.

The **weighted F1-score of 0.4169** is noticeably higher than the **macro F1-score of 0.0865**. That large gap between them indicates that the model performs well only on the majority classes while failing it to learn the minority attack classes. The macro F1-score, which gives equal weight to all classes reveals the true weakness of the model under imbalance conditions.

This pattern is also confirmed in the classification report. Class 0 (majority class, Benign traffic) achieves very high precision (0.98) but low recall (0.35), indicating that while predictions for this class are often correct, a large portion of actual benign samples are misclassified. More critically, most minority classes (1, 2, 3, 5, 7, 8) have precision and recall values close to zero, meaning the model fails to identify these attack types almost entirely.

An exception is class 4 and class 6, which show artificially inflated recall values (0.87 and 1.00 respectively) but very low precision. For this it is suggest that the model is overpredicting these classes, likely due to noise or overlapping feature distributions rather than meaningful learning.

The warning messages about undefined precision further confirm that several classes had no predicted samples at all, reinforcing the fact that the classifier is heavily biased toward a small subset of labels.

Overall, these results demonstrate that the baseline configuration is not suitable for intrusion detection in imbalanced conditions, as it fails to reliably detect most attack categories. This strongly justifies the need for resampling techniques such as SMOTE and Borderline-SMOTE in subsequent experiments.

5 Early Baseline with Minimal Preprocessing

```
print("Running EARLY baseline with minimal preprocessing")

# Copy dataset (avoid modifying original)
merged_df_baseline = merged_df.copy()

# Remove obvious bad rows
merged_df_baseline = merged_df_baseline[merged_df_baseline['Label'] != 'Label']

# Simple missing handling
merged_df_baseline = merged_df_baseline.fillna(0)

# Encode target ONLY
le_baseline = LabelEncoder()
merged_df_baseline['Label_encoded'] = le_baseline.fit_transform(merged_df_baseline['Label'])

# Use ALL numeric features (NO feature selection)
X_baseline = merged_df_baseline.select_dtypes(include=np.number).drop(columns=['Label_encoded'])
y_baseline = merged_df_baseline['Label_encoded']

print(f"Features used: {X_baseline.shape[1]} (no feature selection)")

# Train-test split
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
    X_baseline, y_baseline,
    test_size=0.3,
    random_state=42,
    stratify=y_baseline
)

print("Train-test split complete")

# Train model
rf_baseline_early = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced"
)

rf_baseline_early.fit(X_train_b, y_train_b)

# Predict
y_pred_b = rf_baseline_early.predict(X_test_b)

# Evaluate
accuracy_b = accuracy_score(y_test_b, y_pred_b)
f1_weighted_b = f1_score(y_test_b, y_pred_b, average='weighted')
f1_macro_b = f1_score(y_test_b, y_pred_b, average='macro')

print("\nEARLY BASELINE RESULTS:")
print(f"Accuracy: {accuracy_b:.4f}")
print(f"F1-weighted: {f1_weighted_b:.4f}")
print(f"F1-macro: {f1_macro_b:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_b, y_pred_b))
```

(a)

```

# Evaluate
accuracy_b = accuracy_score(y_test_b, y_pred_b)
f1_weighted_b = f1_score(y_test_b, y_pred_b, average='weighted')
f1_macro_b = f1_score(y_test_b, y_pred_b, average='macro')

print("\nEARLY BASELINE RESULTS:")
print(f"Accuracy: {accuracy_b:.4f}")
print(f"F1-weighted: {f1_weighted_b:.4f}")
print(f"F1-macro: {f1_macro_b:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_b, y_pred_b))

# Save model + results
early_baseline_results = {
    "accuracy": accuracy_b,
    "f1_weighted": f1_weighted_b,
    "f1_macro": f1_macro_b
}

joblib.dump(rf_baseline_early, "rf_early_baseline_model.pkl")
joblib.dump(early_baseline_results, "early_baseline_results.pkl")

print("Early baseline model, results, and encoder saved")

# Clean up RAM
del rf_baseline_early, y_pred_b, early_baseline_results, merged_df_baseline
gc.collect()

print("RAM cleaned after early baseline")

```

(b)

Figure 14. Code of Early Baseline: (a) & (b)

```

Running EARLY baseline with minimal preprocessing
Features used: 1 (no feature selection)
Train-test split complete

EARLY BASELINE RESULTS:
Accuracy: 0.3285
F1-weighted: 0.4169
F1-macro: 0.0865

Classification Report:
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision i
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision i
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision i
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

	precision	recall	f1-score	support
0	0.98	0.35	0.51	2883052
1	0.00	0.00	0.00	85857
2	0.00	0.00	0.00	205804
3	0.00	0.00	0.00	519
4	0.11	0.87	0.20	172857
5	0.00	0.00	0.00	138574
6	0.03	1.00	0.06	41967
7	0.00	0.00	0.00	58008
8	0.00	0.00	0.00	56277
accuracy			0.33	3642915
macro avg	0.13	0.25	0.09	3642915
weighted avg	0.78	0.33	0.42	3642915

```

Early baseline model, results, and encoder saved
RAM cleaned after early baseline

```

Figure 15. Output of Early Baseline Results

4.3 SMOTE Results

The SMOTE sampling strategy significantly increased the representation of minority classes. For example, the smallest class (class 3) increased from 1,211 samples to 2,422 samples, while all other minority classes were also expanded according to the defined sampling strategy. The majority class remained unchanged, preserving the original data distribution in the test set.

SMOTE

The results show a strong improvement compared to the baseline experiment. The **accuracy increased from 0.3285 to 0.9628**, although accuracy alone is not a reliable indicator in imbalanced classification problems.

What is more important is that the **macro F1-score increased from 0.0865 to 0.8140**, indicating a major improvement in the ability to correctly classify minority attack categories. The **weighted F1-score increased from 0.4169 to 0.9616**, showing a strong overall performance across all classes.

This confirms that the detection of the minority classes improved greatly compared to the baseline configuration. In the baseline results most minority classes had an almost near to zero recall what is meaning they were not effectively detected. After SMOTE, several attack categories achieved high recall values (e.g., class 1 ≈ 0.99 , class 3 ≈ 0.92 , class 7 ≈ 0.92), showing that the system can now identify previously underrepresented attack patterns.

However, some imbalance-related challenges remain. Certain classes (such as class 5 and class 6) still show lower precision and recall, indicating overlap between similar attack patterns in the feature space.

SMOTE(SAMPLING)

Generate + Code + Markdown

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    ...
    ...target = int(count * 2)
    ...target = min(target, max_limit)
    ...target = max(target, count)
    ...target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# Apply SMOTE
smote = SMOTE(sampling_strategy=target_samples, random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Class distribution AFTER SMOTE:")
print(pd.Series(y_train_smote).value_counts().sort_index())

# Train Random Forest
rf_smote = RandomForestClassifier(
    ...n_estimators=100,
    ...random_state=42,
    ...n_jobs=-1,
    ...class_weight=None
)
rf_smote.fit(X_train_smote, y_train_smote)

# Save model
joblib.dump(rf_smote, "rf_smote_model.pkl")
print("SMOTE Random Forest trained and saved")

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Predicted classes:", set(y_pred_smote))
```

(a)

```

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Predicted classes:", set(y_pred_smote))

# Evaluate
accuracy = accuracy_score(y_test, y_pred_smote)
f1_weighted = f1_score(y_test, y_pred_smote, average='weighted')
f1_macro = f1_score(y_test, y_pred_smote, average='macro')

print("\nSMOTE Model Evaluation:")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred_smote))

# Save results
smote_results = {
    "accuracy": accuracy,
    "f1_weighted": f1_weighted,
    "f1_macro": f1_macro
}
joblib.dump(smote_results, "smote_results.pkl")
print("SMOTE results saved")

# 8: Clean up memory
del smote, X_train_smote, y_train_smote, rf_smote, y_pred_smote, smote_results
gc.collect()
print("RAM cleaned after SMOTE")

```

(b)

Figure 16. Code of SMOTE: (a) & (b)

```

SMOTE Model Evaluation:
Accuracy: 0.9628
F1-weighted: 0.9616
F1-macro: 0.8140

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.86	0.99	0.92	84693
2	0.72	0.86	0.79	200538
3	0.78	0.92	0.85	519
4	0.98	0.98	0.98	172857
5	0.70	0.48	0.57	130462
6	0.66	0.40	0.50	5838
7	0.72	0.92	0.81	11806
8	0.91	0.95	0.93	35197
accuracy			0.96	3524045
macro avg	0.81	0.83	0.81	3524045
weighted avg	0.96	0.96	0.96	3524045

```

SMOTE results saved
RAM cleaned after SMOTE

```

Figure 17. Output of SMOTE Results

SMOTE Hyperparameter Tuning

To further improve the performance the `k_neighbors` parameter was tuned using values 3, 5, 7, and 9. Each of the configuration was evaluated using the macro F1-score, what was used as the primary selection metric due to its suitability for imbalanced datasets.

The best configuration was achieved with **k = 3**, resulting in a **macro F1-score of 0.8154**, slightly higher than the standard SMOTE configuration (0.8140).

Final evaluation results for the best configuration were:

- Accuracy: 0.9628
- F1-weighted: 0.9616
- F1-macro: 0.8154

SMOTE(SAMPLING) + HYPERPARAMETER TUNING

% Generate + Code + Markdown

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    target = int(count * 2)
    target = min(target, max_limit)
    target = max(target, count)
    target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# SMOTE Hyperparameter Tuning

best_f1 = 0
best_k = None
best_model = None
best_y_pred = None

for k in [3, 5, 7, 9]:
    print(f"\nTesting SMOTE k_neighbors = {k}")

    smote = SMOTE(
        sampling_strategy=target_samples,
        k_neighbors=k,
        random_state=42
    )

    X_res, y_res = smote.fit_resample(X_train, y_train)

    rf = RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1,
        class_weight=None
    )

    rf.fit(X_res, y_res)

    y_pred = rf.predict(X_test)

    print("SMOTE(HYPERPARAMETER) distribution:\n", pd.Series(y_res).value_counts())
    print("Predicted classes:", set(y_pred))

    f1 = f1_score(y_test, y_pred, average='macro')

    print(f"F1-macro: {f1:.4f}")
```

(a)

```

        n_estimators=100,
        random_state=42,
        n_jobs=-1,
        class_weight=None
    )

    rf.fit(X_res, y_res)

    y_pred = rf.predict(X_test)

    print("SMOTE(HYPERPARAMETER) distribution:\n", pd.Series(y_res).value_counts())
    print("Predicted classes:", set(y_pred))

    f1 = f1_score(y_test, y_pred, average='macro')

    print(f"F1-macro: {f1:.4f}")

    if f1 > best_f1:
        best_f1 = f1
        best_k = k
        best_model = rf
        best_y_pred = y_pred

# Save Best Model

joblib.dump(best_model, "rf_smote_tuned_model.pkl")
print("\nBEST MODEL SAVED")

print(f"Best k_neighbors: {best_k}")
print(f"Best F1-macro: {best_f1:.4f}")

# Final Evaluation

accuracy = accuracy_score(y_test, best_y_pred)
f1_weighted = f1_score(y_test, best_y_pred, average='weighted')
f1_macro = f1_score(y_test, best_y_pred, average='macro')

print("\nFINAL RESULTS")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, best_y_pred))

# Save Results

smote_results = {
    # "best_k": best_k,
    "accuracy": accuracy,
    "f1_weighted": f1_weighted,
    "f1_macro": f1_macro
}

joblib.dump(smote_results, "smote_tuned_results.pkl")
print("Results saved")

# Clean Memory

del smote, X_res, y_res, rf, best_model, best_y_pred, smote_results
gc.collect()

print("RAM cleaned")

```

(b)

Figure 18. Code of SMOTE Hyperparameter Tuning: (a) & (b)

```

FINAL RESULTS
Accuracy: 0.9628
F1-weighted: 0.9616
F1-macro: 0.8154

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.86	0.99	0.92	84693
2	0.72	0.86	0.79	200538
3	0.79	0.94	0.85	519
4	0.98	0.98	0.98	172857
5	0.70	0.48	0.57	130462
6	0.66	0.40	0.50	5838
7	0.72	0.92	0.81	11806
8	0.91	0.95	0.93	35197
accuracy			0.96	3524045
macro avg	0.81	0.83	0.82	3524045
weighted avg	0.96	0.96	0.96	3524045

```

Results saved
RAM cleaned

```

Figure 19. Output of SMOTE Hyperparameter Tuning Results

Overall, tuning provided a small but consistent improvement, mainly in balancing performance across classes rather than changing overall predictive strength.

4.4 Borderline-SMOTE Results

The Borderline-SMOTE sampling strategy increased the representation of the minority classes by focusing on to generating synthetic samples near the decision boundaries. The original class imbalance remained highly skewed, with the majority class (class 0) unchanged to preserve the natural distribution of the dataset, while minority classes were expanded according to the defined sampling strategy.

Borderline-SMOTE

The results show a clear improvement compared to the baseline experiment. The accuracy increased from **0.3285 to 0.9598**, although accuracy alone is not a reliable indicator in imbalanced classification problems.

More importantly, the macro F1-score increased from **0.0865 to 0.7794**, indicating a strong improvement in the ability to classify minority attack categories, although slightly lower compared to SMOTE. The weighted F1-score increased from **0.4169 to 0.9588**, showing a strong overall performance across all the classes present in there.

```

~ BORDERLINE SMOTE(SAMPLING)
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    ...
    target = int(count * 2)
    target = min(target, max_limit)
    target = max(target, count)
    target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# Apply Borderline-SMOTE
smote = BorderlineSMOTE(
    ...sampling_strategy=target_samples,
    ...random_state=42,
    ...kind='borderline-1'
)

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Class distribution AFTER Borderline-SMOTE:")
print(pd.Series(y_train_smote).value_counts())

# Train Random Forest
rf_smote = RandomForestClassifier(
    ...n_estimators=100,
    ...random_state=42,
    ...n_jobs=-1,
    ...class_weight=None
)
rf_smote.fit(X_train_smote, y_train_smote)

# Step 4: Save model
joblib.dump(rf_smote, "rf_borderline_smote_model.pkl")
print("Borderline-SMOTE Random Forest trained and saved")

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("BORDERLINE_SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Unique classes in prediction:", len(set(y_pred_smote)))

```

(a)

```

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("BORDERLINE_SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Unique classes in prediction:", len(set(y_pred_smote)))

# Evaluate
accuracy = accuracy_score(y_test, y_pred_smote)
f1_weighted = f1_score(y_test, y_pred_smote, average='weighted')
f1_macro = f1_score(y_test, y_pred_smote, average='macro')

print("\nBorderline-SMOTE Model Evaluation:")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_smote))

# Step 7: Save results
smote_results = {
    .... "accuracy": accuracy,
    .... "f1_weighted": f1_weighted,
    .... "f1_macro": f1_macro
}
joblib.dump(smote_results, "borderline_smote_results.pkl")
print("Borderline-SMOTE results saved")

# Clean up memory
del smote, X_train_smote, y_train_smote, rf_smote, y_pred_smote, smote_results
gc.collect()
print("RAM cleaned after Borderline-SMOTE")

```

(b)

Figure 20. Code of Borderline-SMOTE: (a) & (b)

```

Accuracy: 0.9598
F1-weighted: 0.9588
F1-macro: 0.7794

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.83	0.99	0.91	84693
2	0.73	0.85	0.78	200538
3	0.75	0.95	0.84	519
4	0.98	0.98	0.98	172857
5	0.69	0.48	0.57	130462
6	0.42	0.34	0.38	5838
7	0.67	0.78	0.73	11806
8	0.78	0.93	0.85	35197
accuracy			0.96	3524045
macro avg	0.76	0.81	0.78	3524045
weighted avg	0.96	0.96	0.96	3524045

```

Borderline-SMOTE results saved
RAM cleaned after Borderline-SMOTE

```

Figure 21. Output of Borderline-SMOTE Results

Borderline-SMOTE Hyperparameter Tuning

To further improve performance, the `k_neighbors` and `m_neighbors` parameters were tuned using the following values:

- `k_neighbors` = 3, 5, 7
- `m_neighbors` = 10, 20

Each configuration was evaluated using the macro F1-score, which was used as the primary selection metric due to its suitability for imbalanced datasets.

The best configuration was achieved with `k` = 3 and `m` = 20, resulting in a macro F1-score of **0.7808**, slightly higher than the untuned configuration (0.7794), but still lower than the best SMOTE result.

Final evaluation results for the best configuration were:

- Accuracy: **0.9603**
- F1-weighted: **0.9591**
- F1-macro: **0.7808**

✓ BORDERLINE SMOTE(SAMPLING) + HYPERPARAMETER T

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    ....
    .... target = int(count * 2)
    .... target = min(target, max_limit)
    .... target = max(target, count)
    .... target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# BORDERLINE-SMOTE hyperparameter Tuning

best_f1 = 0
best_params = None
best_model = None
best_pred = None

for k in [3, 5, 7]:
    .... for m in [10, 20]:
        .... print(f"\nTesting k_neighbors={k}, m_neighbors={m}")
        .... smote = BorderlineSMOTE(
        ....     sampling_strategy=target_samples,
        ....     k_neighbors=k,
        ....     m_neighbors=m,
        ....     kind='borderline-1',
        ....     random_state=42
        .... )
        .... X_res, y_res = smote.fit_resample(X_train, y_train)
        .... rf = RandomForestClassifier(
        ....     n_estimators=100,
        ....     random_state=42,
        ....     n_jobs=-1
        .... )
        .... rf.fit(X_res, y_res)
        .... y_pred = rf.predict(X_test)
        .... print("Borderline_SMOTE(HYPERPARAMETER) distribution:\n", pd.Series(y_res).value_counts())
        .... print("Unique predicted classes:", len(set(y_pred)))
        .... f1 = f1_score(y_test, y_pred, average='macro')
```

(a)

```

..... rf.fit(X_res, y_res)

..... y_pred = rf.predict(X_test)
.....
..... print("Borderline SMOTE(HYPERPARAMETER) distribution:\n", pd.Series(y_res).value_counts())
..... print("Unique predicted classes:", len(set(y_pred)))

..... f1 = f1_score(y_test, y_pred, average='macro')

..... print(f"F1-macro: {f1:.4f}")

..... if f1 > best_f1:
.....     best_f1 = f1
.....     best_params = (k, m)
.....     best_model = rf
.....     best_pred = y_pred

# Save best Model

joblib.dump(best_model, "rf_borderline_smote_tuned_model.pkl")

print("\nBEST BORDERLINE-SMOTE MODEL")
print(f"k_neighbors: {best_params[0]}")
print(f"m_neighbors: {best_params[1]}")
print(f"Best F1-macro: {best_f1:.4f}")

# Evaluation

accuracy = accuracy_score(y_test, best_pred)
f1_weighted = f1_score(y_test, best_pred, average='weighted')
f1_macro = f1_score(y_test, best_pred, average='macro')

print("\nFINAL RESULTS")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, best_pred))

# Save Results

results = {
    ....# "best_k_neighbors": best_params[0],
    ....# "best_m_neighbors": best_params[1],
    ...."accuracy": accuracy,
    ...."f1_weighted": f1_weighted,
    ...."f1_macro": f1_macro
}

joblib.dump(results, "borderline_smote_tuned_results.pkl")

# Clean Memory

del smote, X_res, y_res, rf, best_model, best_pred, results
gc.collect()

print("DONE")

```

(b)

Figure 22. Code of Borderline-SMOTE Hyperparameter Tuning: (a) & (b)

```

Accuracy: 0.9603
F1-weighted: 0.9591
F1-macro: 0.7808

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.82	1.00	0.90	84693
2	0.73	0.85	0.78	200538
3	0.76	0.93	0.84	519
4	0.98	0.98	0.98	172857
5	0.70	0.48	0.57	130462
6	0.40	0.39	0.39	5838
7	0.67	0.73	0.70	11806
8	0.83	0.92	0.88	35197
accuracy			0.96	3524045
macro avg	0.76	0.81	0.78	3524045
weighted avg	0.96	0.96	0.96	3524045

```

DONE

```

Figure 23. Output of Borderline-SMOTE Hyperparameter Tuning Results

Overall, tuning provided only a marginal improvement, mainly in stabilizing performance across classes rather than significantly increasing predictive strength. However, Borderline-SMOTE remained less effective than standard SMOTE in terms of overall macro-level performance.

4.5 Experimental Results Comparison

The performance of all of the five experimental configurations was evaluated using their accuracy, weighted F1-score, and macro F1-score to assess both overall performance and class-wise balance under severe class imbalance conditions.

The **early baseline** configuration achieved an accuracy of **0.3285**, but its macro F1-score was extremely low at **0.0865**, indicating very poor performance on minority attack classes. The weighted F1-score of **0.4169** shows that the model was heavily biased toward the majority class (benign traffic), correctly classifying it but failing to detect most attack types. Based on that result it confirms that accuracy alone is misleading in imbalanced intrusion detection scenarios.

The introduction of **SMOTE significantly improved performance**, increasing accuracy to **0.9628** and macro F1-score to **0.8140**. This represents a substantial improvement in minority class detection, where recall values for several attack categories increased from near zero in the baseline to above 0.85–0.99 in multiple classes. The weighted F1-score also increased to **0.9616**, indicating strong overall predictive capability while maintaining balance across classes.

With the **SMOTE + hyperparameter tuning** configuration the performance improved slightly even further, achieving the best overall macro F1-score of **0.8154**, while at the same time the accuracy and weighted F1-score remained stable at **0.9628** and **0.9616** respectively. This indicates that tuning improved class balance marginally rather than overall accuracy.

The **Borderline-SMOTE** configuration achieved an accuracy of **0.9598** and a macro F1-score of **0.7794**, which is lower than SMOTE. Although it improved boundary-based learning for minority classes, it underperformed in comparison for several attack categories, particularly those with overlapping feature distributions, where recall dropped (e.g., class 6 with recall ≈ 0.34).

After tuning the **Borderline-SMOTE configuration achieved a macro F1-score of 0.7808**, showing only a marginal improvement. However, it still remained below SMOTE-based configurations in both class-wise balance and overall detection capability.



Figure 24. Confusion matrix of SMOTE Hyperparameter Tuning

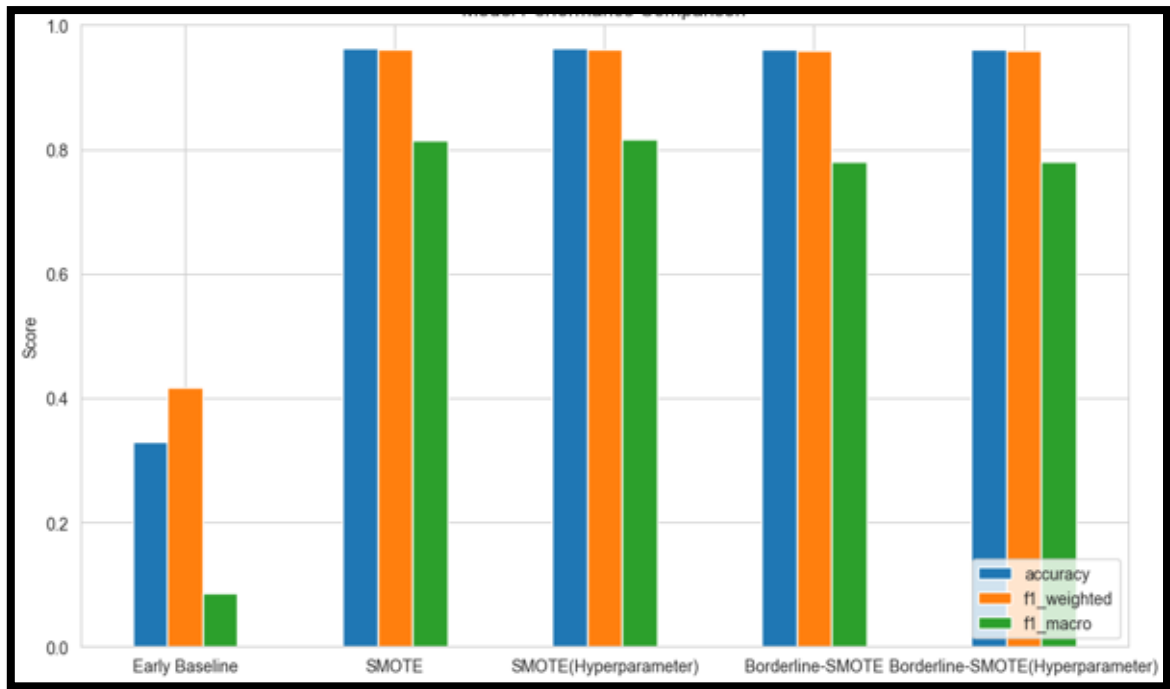


Figure 25. Comparison of all the 5 configurations

9 Model Evaluation

- > Load all results
 - ▷ 1 cell hidden ...
- > Create comparison table
 - ▷ 1 cell hidden ...
- > Visual comparison
 - ▷ 1 cell hidden ...
- > Automatically select best model
 - ▷ 1 cell hidden ...
- > Select + save final model
 - ▷ 1 cell hidden ...
- > Add Confusion Matrix
 - ▷ 1 cell hidden ...
- ~ Test real predictions


```
best_model = joblib.load("final_nids_model.pkl")
le = joblib.load("label_encoder.pkl")

sample = X_test.iloc[[0]]

pred = best_model.predict(sample)[0]
true = y_test.iloc[0]

print("True label (encoded):", true)
print("Predicted label (encoded):", pred)

print("True label (text):", le.inverse_transform([true])[0])
print("Predicted label (text):", le.inverse_transform([pred])[0])
```

Python

Figure 26. Test for predictions of SMOTE Hyperparameter Tuning

```
True label (encoded): 0
Predicted label (encoded): 0
True label (text): Benign
Predicted label (text): Benign
```

Figure 27. Output Test for predictions of SMOTE Hyperparameter Tuning

4.6 Discussion

The results clearly demonstrate that class imbalance has a strong impact on machine learning-based intrusion detection systems. The baseline Random Forest configuration achieved an accuracy of **0.3285**, however this is misleading due to its strong bias toward the majority class (benign traffic). This is further confirmed by its extremely low macro F1-score of **0.0865**, showing that minority attack classes were largely not correctly identified. This validates that accuracy alone is not a reliable evaluation metric for highly imbalanced cybersecurity datasets.

In contrast for it the best-performing configuration was the **SMOTE configuration combined with hyperparameter tuning**, what is achieved an accuracy of **0.9628** and a macro F1-score of **0.8154**. The high macro F1-score indicates a strong balance between majority and minority class performance, making it the most appropriate metric for this problem. Therefore, **macro F1-score is the preferred evaluation metric**, as it treats all classes equally and prevents majority-class dominance from hiding poor detection performance on rare attacks.

These misclassifications indicate that while resampling improves class balance, feature representation still limits full separability between certain attack types. Flow-based network features alone may not be sufficient to fully distinguish fine-grained attack behaviours, especially in cases where attacks share similar traffic signatures.

From a real-world perspective, this has important implications for intrusion detection systems. A model biased like the baseline would fail in operational environments because it would miss most real attacks, creating serious security vulnerabilities. In contrast, the SMOTE-based configuration significantly improves detection of rare but critical threats, making it far more suitable for deployment in enterprise or cloud network monitoring systems. However also worth to mention that residual confusion between similar attack types can suggest it that further improvements such as feature engineering or deep learning approaches may be required for production-level robustness.

Chapter 5: Conclusion

5.1 Summary of the Research

This study investigated the application of machine learning techniques for detecting and classifying network intrusion traffic of the CICID2018 dataset and the merged and selected version of subsets of it. The focus was on identify and be able to differentiate between benign network traffic and multiple attack types such as brute-force, denial-of-service (DoS), distributed denial-of-service (DDoS) and botnet activity.

A Random Forest classifier was developed and evaluated as the main model. The research placed particular emphasis on the impact of class imbalance and the effectiveness of resampling techniques, specifically SMOTE and Borderline-SMOTE, in improving classification performance.

5.2 Achievement of Objectives

All research objectives outlined in Chapter 1 were successfully achieved:

- In the CICIDS2018 dataset related subsets selected and prepared for further analysis.
- Data preprocessing was performed such as cleaning, transformation and feature selection of it.
- A Random Forest classifier was implemented and evaluated.
- Class imbalance handling techniques SMOTE and Borderline-SMOTE were applied and compared, including tuned variations of those configurations.
- Model performance was analysed using multiple evaluation metrics, including macro F1-score.

5.3 Research Questions Answered

Can machine learning models distinguish between benign and malicious network traffic?

Yes. The result of this study highlights that machine learning models, particularly Random Forest, could effectively differentiate network traffic between benign and multiple attack categories.

Does handling class imbalance improve detection performance?

Yes. Both SMOTE and Borderline-SMOTE improved detection of minority attack classes. SMOTE produced the most consistent and balanced improvement across evaluation metrics.

Which resampling technique performs best?

SMOTE combined it with hyperparameter tuning produced the best overall performance, achieved the highest macro F1-score and delivered the most balanced classification performance. Borderline-SMOTE improved boundary learning but did not outperform SMOTE.

5.4 Hypothesis Evaluation

The hypothesis stated that machine learning models, particularly Random Forest classifiers, could detect network attack traffic effectively and that their performance can be improved through class imbalance handling techniques such as SMOTE and Borderline-SMOTE.

The results of this study or experiment support this hypothesis. While the baseline configuration achieved a good accuracy compared to the fact that it is just a baseline configuration, it performed poorly on minority classes thanks to the severe class imbalance. Both SMOTE and Borderline-SMOTE improved the performance by improving minority class detection and overall balance.

However, the most effective configuration was the combination of SMOTE with hyperparameter tuning of the Random Forest classifier. This method achieved the highest macro F1-score and highlighted the most balanced performance across all evaluation metrics.

Therefore, the hypothesis is accepted, as both resampling techniques and configuration optimisation contributed to improve the intrusion detection performance together and how SMOTE combined with hyperparameter tuning made it possible to get the best overall results.

5.5 Contribution to the Field

This study contributes to the field of intrusion detection systems by providing a structured comparative evaluation of class imbalance handling techniques in a multi-class network traffic classification problem.

Firstly, it highlighted the effectiveness of Random Forest classifiers on the CICIDS2018 data subsets for differentiate between benign and multiple attack categories proving its suitable for structured network flow data.

Secondly, it provides an empirical comparison between SMOTE and Borderline-SMOTE under identical experimental conditions, including tuned configurations. The results show that while both techniques improve minority class detection, SMOTE achieves the most consistent overall performance, particularly in terms of macro F1-score.

Thirdly, the limitations of accuracy as a performance metric in imbalanced cybersecurity datasets and reinforces the importance of macro-averaged F1-score as a more reliable evaluation measure for intrusion detection systems also identified.

Finally, the study demonstrates that combining data level techniques for balancing like SMOTE with configuration level optimisation (hyperparameter tuning) will produce the best overall performance, both data preprocessing and model configuration are crucial to be able to improve IDS effectiveness.

5.6 Final Remarks

The study produced important findings what confirm that machine learning techniques can be effectively used for intrusion detection tasks when it is combined with an appropriate preprocessing and class imbalance handling method.

However, the results also highlight a key challenge: class imbalance significantly affects model reliability in detecting rare attack types. Without addressing this issue, models tend to become biased toward benign traffic, reducing their practical usefulness in real-world cybersecurity environments.

Overall, SMOTE was identified as the most effective resampling technique for this dataset, producing the most balanced and reliable classification performance compared to both the baseline and Borderline-SMOTE approaches.

Chapter 6: Future Work

6.1 Real-Time Deployment in Virtual Lab Environments

While this study highlighted and demonstrated strong performance in an offline dataset setup environment for future work, it should focus on to test those configurations in real-time scenarios. This would give a more realistic assessment of its effectiveness in practical cybersecurity settings.

A suitable approach would be to test it within a controlled virtual lab environment using platforms such as Kali Linux, Metasploitable, DVWA or OWASP Juice Shop. The network traffic could be generated and captured using tools such as Wireshark or tcpdump, this would allow the model to test if it will be able to differentiate between benign and attack labels in live traffic rather than in a pre-collected dataset.

The mentioned setup and testing will help to validate if the model maintains performance under a dynamic environment and unpredictable network conditions too or not.

6.2 Development of a Real-Time Intrusion Detection System

An important extension of this work would be the development of a real-time intrusion detection system (IDS). Instead of relying on static datasets such as the main concept of this study the model could be integrated into a live monitoring pipeline capable of continuously analysing network traffic.

Such a system could generate real-time alerts when malicious activity is detected, significantly increasing its practical value in operational cybersecurity environments.

6.3 Feature Engineering Improvements

Future research should focus on to improve classification performance by expanding and refining the feature set used for training.

Additional flow-based, packet-level, and time-based features could be able to help improve differentiation between similar attack types such as DoS and DDoS traffic.

Techniques such as recursive feature elimination, mutual information ranking, or automated feature selection methods could also be explored to optimise model input features.

6.4 Exploration of Advanced Machine Learning Models

Although Random Forest performed really well in this study, future work could also focus on to investigate more advanced machine learning approaches such as:

- Gradient boosting models such as XGBoost and LightGBM.
- Deep learning models such as feedforward neural networks or recurrent neural networks.

The mentioned models above may be better to capture complex, and non-linear relationships in network traffic data and also could potentially improve classification accuracy further.

6.5 Evaluation on Additional Datasets

Future research should also evaluate the model on additional intrusion detection datasets or combine multiple datasets all together in order for training and testing purpose.

This would improve generalisation and ensure that the model is not overfitted to a single dataset structure such as CICIDS2018.

Testing it on multiple datasets not just the one this study focused on would also help validate robustness in different network environments and attack scenarios.

6.6 Summary

Overall, while this study highlighted and demonstrated that machine learning combined it with techniques for handling imbalanced classes can effectively detect network intrusion traffic, there remains significant scope for improvement.

Future developments focusing on real time deployment, advanced modelling techniques, improved feature engineering, and explainability would significantly improve and enhance it further for the practical applicability of intrusion detection systems in real-world cybersecurity environments.

References

- Abbasi, M. Shahraki, A. and Taherkordi, A. (2021). ‘Deep Learning for Network Traffic Monitoring and Analysis (NTMA): A Survey’, *Computer Communications*, 170(March), pp. 19-41. <https://doi.org/10.1016/j.comcom.2021.01.021>
- Ali, M.L., Thakur, K., Schmeelk, S., Debellio, J. and Dragos, D. (2025). ‘Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study’, *Applied Science*, 15(4), pp. 1-19. <https://doi.org/10.3390/app15041903>
- Amachaghi, E. Abdulkareem, S., Foh, Ch., Mi, D. and Shojafar, M. (2025). ‘Improving intrusion detection in O-RAN with synthetic data generation: A GAN and SMOTE approach’, *Telematics and Informatics Reports*, 20, pp. 1-10. <https://doi.org/10.1016/j.teler.2025.100269>
- Buczak, A.L. and Guven, E. (2016). ‘A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection’, *IEEE Communications Surveys & Tutorials*, 18(2), pp. 1153-1176. DOI: [10.1109/COMST.2015.2494502](https://doi.org/10.1109/COMST.2015.2494502)
- Canadian Institute for Cybersecurity. *CSE-CIC-IDS2018 on AWS*. Available at: <https://www.unb.ca/cic/datasets/ids-2018.html> (Accessed: 10 April 2026)
- Canadian Institute for Cybersecurity. *Intrusion detection evaluation dataset (CIC-IDS2017)*. Available at: <https://www.unb.ca/cic/datasets/ids-2017.html> (Accessed: 10 April 2026)
- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.Ph. (2002). ‘SMOTE: Synthetic Minority Over-sampling Technique’, *Journal of Artificial Intelligence Research*, 16, 321-357. <https://doi.org/10.1613/jair.953>
- Diana, L., Dini, P. and Paolini, D. (2025). ‘Overview on Intrusion Detection Systems for Computers Networking Security’, *Computers*, 14(3), pp. 1-44. <https://doi.org/10.3390/computers14030087>
- Sithole, Th. and Du Toit, J. (2022). ‘A Cyber Counterintelligence Competence Framework’, in Eze, Th., Khan, N. and Onwubiko, C. (eds). *ECCWS 2022 21st European Conference on Cyber Warfare and Security*. England: University of Chester, pp. 368-377.
- García-Teodoro, P., Díaz-Verdejo, J., Maciá-Fernández, G. and Vázquez, E. (2009). ‘Anomaly-based network intrusion detection: Techniques, systems and challenges’, *Computers & Security*, 28(1-2), pp. 18-28. <https://doi.org/10.1016/j.cose.2008.08.003>

- Genuario, F., Santoro, G., Giliberti, M., Bello, S., Zazzera, E. and Impedovo, D. (2024). ‘Machine Learning-Based Methodologies for Cyber-Attacks and Network Traffic Monitoring: A Review and Insights’, *Information* 15(11), pp. 1-27, <https://doi.org/10.3390/info15110741>
- Han, D., Li, H., Fu, X. and Z, Sh. (2024). ‘Traffic Feature Selection and Distributed Denial of Service Attack Detection in Software-Defined Networks Based on Machine Learning’, *Sensors* 24(13), pp. 1-22. <https://doi.org/10.3390/s24134344>
- Han, H., Wang, W.Y. and Mao, B.H. (2005). ‘Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning’, in Huang, D.S., Zhang, X.P. and Huang, G.B. (eds). *Advances in Intelligent Computing. ICIC 2005. Lecture Notes in Computer Science*, vol 3644. Springer: Berlin, Heidelberg. https://doi.org/10.1007/11538059_91
- Hindy, H., Atkinson, R. Tachtatzis, Ch., Bayne, E., Bures, M. and Bellekens, X. (2021). ‘Utilising Flow Aggregation to Classify Benign Imitating Attacks’, *Sensors*, 21(5), pp. 1-17. <https://doi.org/10.3390/s21051761>
- Hnamte, V. and Hussain, J. (2023). ‘Dependable intrusion detection system using deep convolutional neural network: A Novel framework and performance evaluation approach’, *Telematics and Informatics Reports*, 11(September). <https://doi.org/10.1016/j.teler.2023.100077>
- Liao, H.J., Lin, Ch.H., Lin, Y.Ch. and Tung, K.Y. (2013). ‘Intrusion detection system: A comprehensive review’, *Journal of Network and Computer Applications*, 36(1), pp. 16-24. <https://doi.org/10.1016/j.jnca.2012.09.004>
- Mahdavifar, S. and Ghorbani, A.A. (2019). ‘Application of deep learning to cybersecurity: A survey’, *Neurocomputing*, 347, pp.149–176. <https://doi.org/10.1016/j.neucom.2019.02.056>
- Mittal, P., Lallie, H.S. Titis, E. (2025). ‘Impact of imbalanced datasets on ML algorithms for malware classification’, *Information Security Journal: A Global Perspective*, 34(3), pp. 251-264. <https://doi.org/10.1080/19393555.2025.2459736>
- Saputra, F., Ilham, I., Rizal, M., Wisda, W., Wanita, F., Mursalim, M. and Fadillah, A. (2025). ‘Enhancing Intrusion Detection Using Random Forest and SMOTE on the NSL-KDD Dataset’, *Journal of System and Computer Engineering* 6(3), pp. 240–247. <https://doi.org/10.61628/jsce.v6i3.2056>

Sarker, I.H., Kayes, A.S.M., Badsha, Sh., Alqahtani, H., Watters P. and Ng, A. (2020). ‘Cybersecurity data science: an overview from machine learning perspective’, *Journal of Big Data*, 7(41), pp. 1-29. <https://doi.org/10.1186/s40537-020-00318-5>

Sharafaldin, I., Lashkari, A. and Ghorbani, A. (2018) ‘Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization’, *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, pp. 108-116. DOI: 10.5220/0006639801080116

Thejas, G.S., Hariprasad, Y., Iyengar, S.S., Sunitha, N.R., Badrinath, P. and Chennupati, Sh. (2022). ‘An extension of Synthetic Minority Oversampling Technique based on Kalman filter for imbalanced datasets’, *Machine Learning with Applications*, 8(June), pp. 1-12. <https://doi.org/10.1016/j.mlwa.2022.100267>

Tsai, Ch.F., Hsu, Y.F., Lin, C.Y. and Lin, W.Y. (2009). ‘Intrusion detection by machine learning: A review’, *Expert Systems with Applications*, 36(10), pp. 11994-12000. <https://doi.org/10.1016/j.eswa.2009.05.029>

Widodo, A.O., Setiawan, B. Indraswari, R. (2024). ‘Machine Learning-Based Intrusion Detection on Multi-Class Imbalanced Dataset Using SMOTE’, *Procedia Computer Science*, 234, pp. 578–583. <https://doi.org/10.1016/j.procs.2024.03.042>

Wu, T., Fan, H., Zhu, H., You, C., Zhou, H. and Huang, X. (2022). ‘Intrusion detection system combined enhanced random forest with SMOTE algorithm’, *EURASIP Journal on Advances in Signal Processing*, pp. 1-20. <https://doi.org/10.1186/s13634-022-00871-6>

Photos References:

Schultz, K. (2025). ‘Understanding the Role of Knowledge Intelligence in the CRISP-DM Framework: A Guide for Data Science Projects’. Available at: <https://enterprise-knowledge.com/understanding-the-role-of-knowledge-intelligence-in-the-crisp-dm-framework-a-guide-for-data-science-projects/>

Further Studies:

Kaggle IDS 2017 (08/03/2026): <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset>

Kaggle IDS 2018 (08/03/2026): <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv>

Data Science CRISP-DM (09/03/2026): <https://www.datascience-pm.com/crisp-dm-2/>

Science Direct Enhance SMOTE (09/03/2026): <https://www.sciencedirect.com/science/article/pii/S2666827024000732>

Geeksforgeeks SMOTE (11/03/2026): <https://www.geeksforgeeks.org/machine-learning/smote-for-imbalanced-classification-with-python/>

Geeksforgeeks Feature Selection Techniques (12/03/2026): <https://www.geeksforgeeks.org/machine-learning/feature-selection-techniques-in-machine-learning/>

Medium Machine Learning (13/03/2026): <https://medium.com/@abdallahashraf90x/all-you-need-to-know-about-correlation-for-machine-learning-e249fec292e9>

Scikit-learn Feature Selection (13/03/2026): https://scikit-learn.org/stable/modules/feature_selection.html

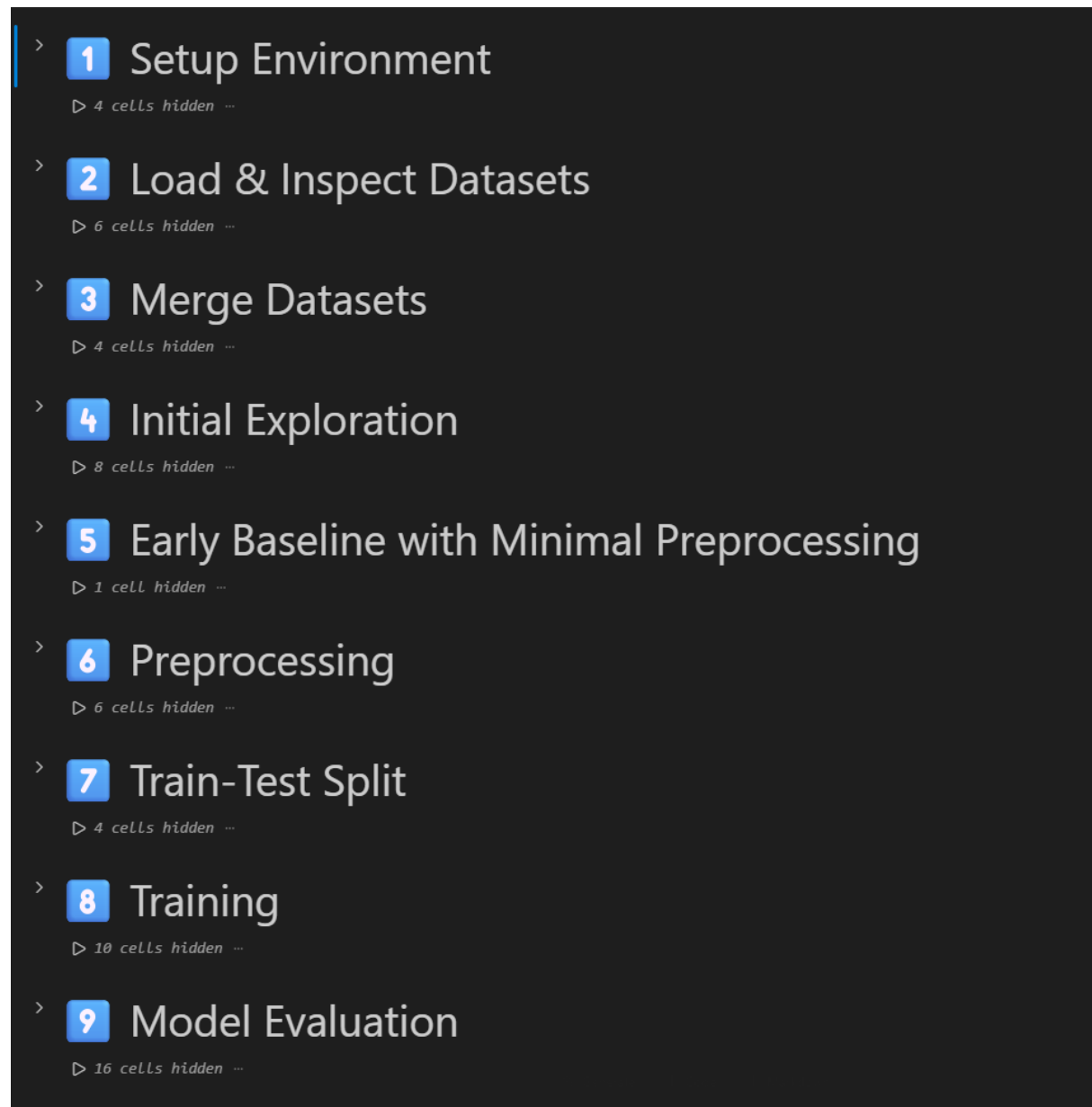
Geeksforgeeks Random Forest (15/03/2026): <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>

Towards Data Science SMOTE (16/03/2026): <https://towardsdatascience.com/class-imbalance-smote-borderline-smote-adasync-6e36c78d804/>

Github scikit-learn (16/03/2026): https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/feature_selection/_univariate_selection.py

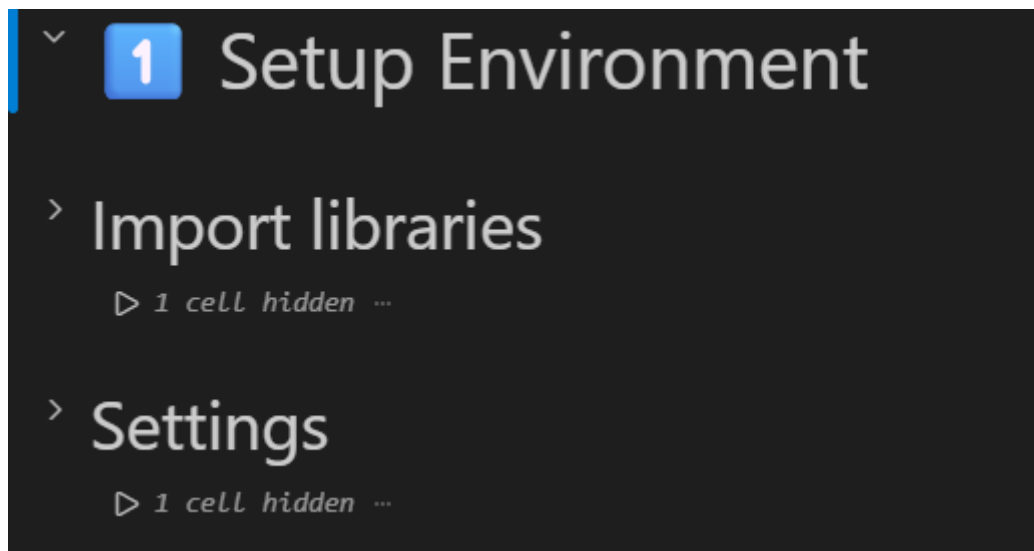
<https://github.com/scikit-learn-contrib/imbalanced-learn>

Appendix

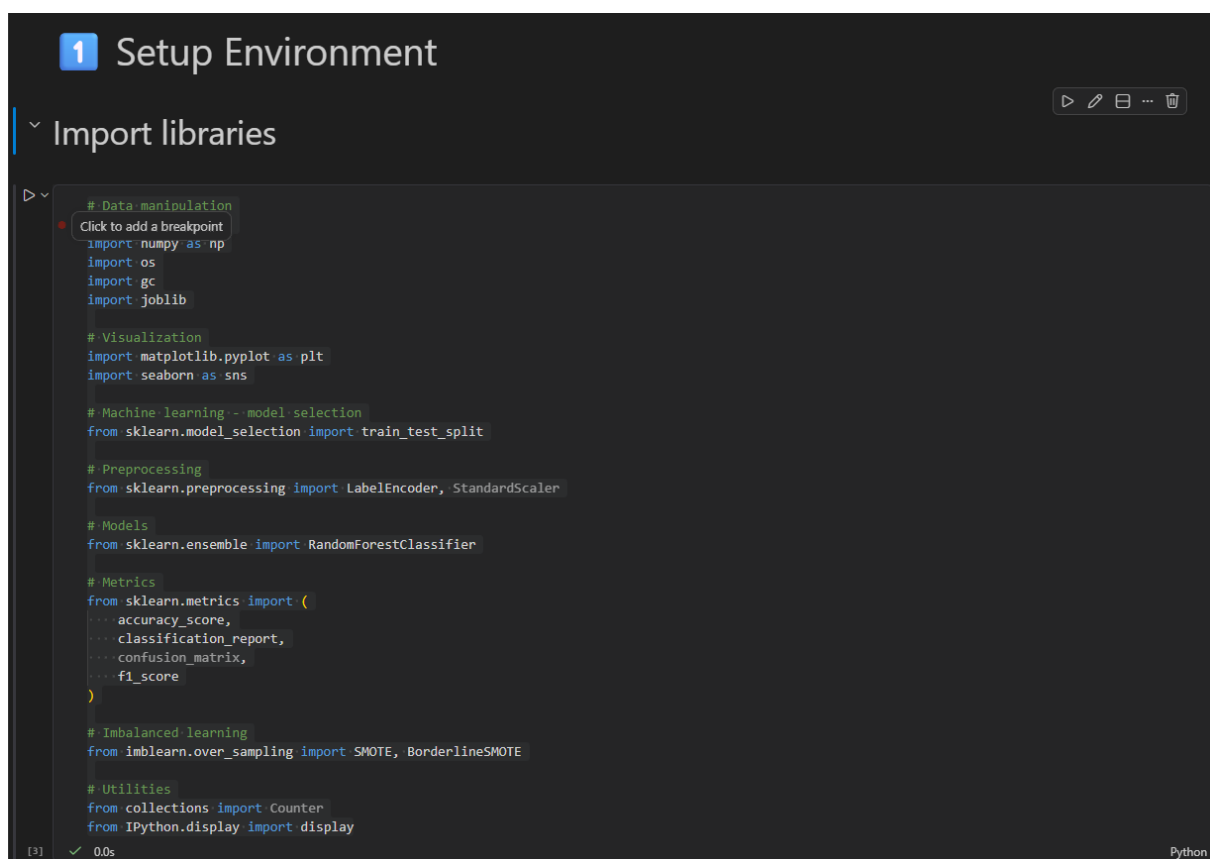


- > **1** Setup Environment
▷ 4 cells hidden ...
- > **2** Load & Inspect Datasets
▷ 6 cells hidden ...
- > **3** Merge Datasets
▷ 4 cells hidden ...
- > **4** Initial Exploration
▷ 8 cells hidden ...
- > **5** Early Baseline with Minimal Preprocessing
▷ 1 cell hidden ...
- > **6** Preprocessing
▷ 6 cells hidden ...
- > **7** Train-Test Split
▷ 4 cells hidden ...
- > **8** Training
▷ 10 cells hidden ...
- > **9** Model Evaluation
▷ 16 cells hidden ...

Main Structure



Setup Environment



Import libraries

1 Setup Environment

> Import libraries

▶ 1 cell hidden ...

Settings

```
# Pandas display options
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
pd.set_option('display.float_format', '{:.3f}'.format)

# Set random seed for reproducibility
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

# Visualization style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

# Confirmation message
print("Display settings configured and random seed set.")
```

[2]

✓ 0.0s

Python

Settings

2 Load & Inspect Datasets

> List all dataset files

▶ 1 cell hidden ...

> Inspect each dataset

▶ 1 cell hidden ...

> Preview each dataset Label column

▶ 2 cells hidden ...

Load & Inspect Datasets

2 Load & Inspect Datasets

~ List all dataset files

```
# Define the folder containing the datasets
dataset_folder = "Dataset"

# List all dataset files
dataset_files = [f for f in os.listdir(dataset_folder) if f.endswith('.csv')]
print("Found dataset files:")
for file in dataset_files:
    print(f" - {file}")
print("Dataset files listed successfully.")
```

[3]

Python

List all dataset files

```
Found dataset files:
- 02-14-2018.csv
- 02-15-2018.csv
- 02-16-2018.csv
- 02-20-2018.csv
- 02-21-2018.csv
- 02-22-2018.csv
- 02-23-2018.csv
- 02-28-2018.csv
- 03-01-2018.csv
- 03-02-2018.csv
Dataset files listed successfully.
```

List all dataset files output

~ Inspect each dataset

```
for file in dataset_files:
    print("="*80)
    print(f"Dataset: {file}")
    print("="*80)

    # Load dataset
    df = pd.read_csv(os.path.join(dataset_folder, file))

    # Preview first 5 rows
    print("\nHead (first 5 rows):")
    display(df.head())

    # Info: columns, non-null counts, dtypes
    print("\nInfo:")
    display(pd.DataFrame(df.dtypes, columns=['Data Type']))

    # Basic statistics for numeric columns
    print("\nDescribe (numeric stats):")
    display(df.describe())

    # Missing values
    print("\nMissing Values:")
    missing = df.isnull().sum()
    missing = missing[missing > 0]
    if missing.empty:
        print("No missing values")
    else:
        display(missing.to_frame('Missing Count'))

    print("Inspection complete for this dataset.\n\n")
```

[4]

Python

Inspect each dataset

Dataset: 02-14-2018.csv

Head (first 5 rows):

	Dst Port	Protocol	Timestamp	Flow Duration	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Fwd Pkt Len Max	Fwd Pkt Len Min	Fwd Pkt Len Mean	Fwd Pkt Len Std	Bwd Pkt Len Max	Bwd Pkt Len Min	Bwd Pkt Len Mean	Bwd Pkt Len Std	Flow Byts/s	Flow Pkts/s	Flow IAT Mean
0	0	0	14/02/2018 08:31:01	112641719	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56320859.500
1	0	0	14/02/2018 08:33:50	112641466	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56320733.000
2	0	0	14/02/2018 08:36:39	112638623	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56319311.500
3	22	6	14/02/2018 08:40:13	6453966	15	10	1239	2273	744	0	82.600	196.741	976	0	227.300	371.678	544.162	3.874	268915.250
4	22	6	14/02/2018 08:40:23	8804066	14	11	1143	2209	744	0	81.643	203.746	976	0	200.818	362.250	380.733	2.840	366836.083

Info:

Data Type	
Dst Port	int64
Protocol	int64
Timestamp	str
Flow Duration	int64
Tot Fwd Pkts	int64
Tot Bwd Pkts	int64
TotLen Fwd Pkts	int64
TotLen Bwd Pkts	int64
Fwd Pkt Len Max	int64
Fwd Pkt Len Min	int64
Fwd Pkt Len Mean	float64
Fwd Pkt Len Std	float64
Bwd Pkt Len Max	int64
Bwd Pkt Len Min	int64
Bwd Pkt Len Mean	float64
Bwd Pkt Len Std	float64
Flow Byts/s	float64
Flow Pkts/s	float64
Flow IAT Mean	float64
Flow IAT Std	float64
Flow IAT Max	int64
Flow IAT Min	int64
Fwd IAT Tot	int64
Fwd IAT Mean	float64
Fwd IAT Std	float64
Fwd IAT Max	int64

init Bwd Win Byts

int64

Fwd Act Data Pkts

int64

Fwd Seg Size Min

int64

Active Mean

float64

Active Std

float64

Active Max

int64

Active Min

int64

Idle Mean

float64

Idle Std

float64

Idle Max

int64

Idle Min

int64

Label

str

Describe (numeric stats):

```
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pandas\core\nanops.py:1020: RuntimeWarning: invalid value encountered in sub
sqr = _ensure_numeric((avg - values) ** 2)
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pandas\core\nanops.py:1020: RuntimeWarning: invalid value encountered in sub
sqr = _ensure_numeric((avg - values) ** 2)
```

	Dst Port	Protocol	Flow Duration	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Fwd Pkt Len Max	Fwd Pkt Len Min	Fwd Pkt Len Mean	Fwd Pkt Len Std
count	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000	1048575.000
mean	4876.262	8.108	6255554.609	6.207	7.211	447.994	4521.803	174.574	8.390	38.796	50.211
std	14443.436	4.461	1260291108.633	44.479	104.868	15735.415	151502.121	287.671	19.483	53.319	85.189
min	0.000	0.000	-919011000000.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
25%	22.000	6.000	7.000	1.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000
50%	53.000	6.000	1023.000	2.000	1.000	36.000	55.000	34.000	0.000	25.667	0.000
75%	443.000	6.000	406669.000	7.000	6.000	455.000	768.000	199.000	0.000	55.500	92.953
max	65533.000	17.000	119999989.000	5115.000	9198.000	8591554.000	13397730.000	64440.000	1460.000	11217.025	15760.838

Missing Values:

	Missing Count
Flow Byts/s	2277

Inspection complete for this dataset.

Dataset: 02-15-2018.csv

Head (first 5 rows):

	Dst Port	Protocol	Timestamp	Flow Duration	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Fwd Pkt Len Max	Fwd Pkt Len Min	Fwd Pkt Len Mean	Fwd Pkt Len Std	Bwd Pkt Len Max	Bwd Pkt Len Min	Bwd Pkt Len Mean	Bwd Pkt Len Std	Flow Byts/s	Flow Pkts/s
0	0	0	15/02/2018 08:25:18	112641158	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027
1	22	6	15/02/2018 08:29:05	37366762	14	12	2168	2993	712	0	154.857	254.855	976	0	249.417	395.929	138.117	0.696

Inspect each dataset output

2 Load & Inspect Datasets

› List all dataset files

▷ 1 cell hidden ...

› Inspect each dataset

▷ 1 cell hidden ...

~ Preview each dataset Label column

```
label_info = {}

for file in dataset_files:
    file_path = os.path.join(dataset_folder, file)
    df = pd.read_csv(file_path, low_memory=False)

    if 'Label' in df.columns:
        value_counts = df['Label'].value_counts(normalize=True) * 100
        label_info[file] = {
            'Label_column_present': True,
            'unique_values_count': df['Label'].nunique(),
            'unique_values_names': df['Label'].unique(),
            'value_percentages': value_counts.to_dict()
        }
    else:
        label_info[file] = {
            'Label_column_present': False,
            'unique_values_count': 0,
            'unique_values_names': [],
            'value_percentages': {}
        }

# Display summary
for file, info in label_info.items():
    print(f"{file}:")
    print(f"  Label column present = {info['Label_column_present']}")
    print(f"  Unique values count = {info['unique_values_count']}")
    print(f"  Unique value names = {info['unique_values_names']}")
    print(f"  Value percentages:")
    for label, pct in info['value_percentages'].items():
        print(f"    {label}: {pct:.2f}%")
    print("-" * 50)

print("Label column inspection completed with percentages.")
```

[5]

Python

Preview each dataset Label column

```
Unique value names = <StringArray>
['Benign', 'Label', 'Infiltration']
Length: 3, dtype: str
Value percentages:
  Benign: 88.76%
  Infiltration: 11.23%
  Label: 0.01%
-----
03-01-2018.csv:
  Label column present = True
  Unique values count = 3
  Unique value names = <StringArray>
['Benign', 'Label', 'Infiltration']
Length: 3, dtype: str
Value percentages:
  Benign: 71.89%
  Infiltration: 28.11%
  Label: 0.01%
-----
03-02-2018.csv:
  Label column present = True
  Unique values count = 2
  Unique value names = <StringArray>
['Benign', 'Bot']
Length: 2, dtype: str
Value percentages:
  Benign: 72.71%
  Bot: 27.29%
-----
Label column inspection completed with percentages.
```

Preview each dataset Label column output

3 Merge Datasets

> Load each selected dataset

▶ 1 cell hidden ...

> Merge, Save, and Free Memory

▶ 1 cell hidden ...

Merge Datasets

3 Merge Datasets

> Load each selected dataset

```
# Load selected datasets into a list of DataFrames
selected_files = [
    '02-14-2018.csv', # FTP-BruteForce / SSH-Bruteforce
    '02-16-2018.csv', # DoS attacks-Hulk
    '02-20-2018.csv', # DDoS attacks-LOIC-HTTP
    '02-21-2018.csv', # DDoS attacks-HOIC
    '03-02-2018.csv' # Bot
]

dataframes = []

for file in selected_files:
    file_path = os.path.join(dataset_folder, file)
    df = pd.read_csv(file_path, low_memory=False)
    dataframes.append(df)
    print(f"{file} loaded, shape: {df.shape}")

print("All selected datasets loaded into memory as separate DataFrames.")
```

[6]

Python

Load each selected dataset

```
02-14-2018.csv loaded, shape: (1048575, 80)
02-16-2018.csv loaded, shape: (1048575, 80)
02-20-2018.csv loaded, shape: (7948748, 84)
02-21-2018.csv loaded, shape: (1048575, 80)
03-02-2018.csv loaded, shape: (1048575, 80)
All selected datasets loaded into memory as separate DataFrames.
```

Load each selected dataset output

3 Merge Datasets

> Load each selected dataset

▷ 1 cell hidden ...

✓ Merge, Save, and Free Memory

```
# Merge all selected datasets into a single DataFrame
merged_df = pd.concat(dataframes, ignore_index=True)
print(f'Datasets merged successfully. Merged shape: {merged_df.shape}')

# Preview first 5 rows
display(merged_df.head())

# Save merged dataset for future use
merged_df.to_csv("merged_dataset.csv", index=False)
print("Merged dataset saved as 'merged_dataset.csv'.")

# Free memory by deleting individual DataFrames
del dataframes
print("Individual DataFrames cleared from memory.")
```

[7]

Python

Merge, Save, and Free Memory

Datasets merged successfully. Merged shape: (12143048, 84)

	Dst Port	Protocol	Timestamp	Flow Duration	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Fwd Pkt Len Max	Fwd Pkt Len Min	Fwd Pkt Len Mean	Fwd Pkt Len Std	Bwd Pkt Len Max	Bwd Pkt Len Min	Bwd Pkt Len Mean	Bwd Pkt Len Std	Flow Byts/s	Flow Pkts/s	Flow IAT Mean
0	0	0	14/02/2018 08:31:01	112641719	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56320859.500
1	0	0	14/02/2018 08:33:50	112641466	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56320733.000
2	0	0	14/02/2018 08:36:39	112638623	3	0	0	0	0	0	0.000	0.000	0	0	0.000	0.000	0.000	0.027	56319311.500
3	22	6	14/02/2018 08:40:13	6453966	15	10	1239	2273	744	0	82.600	196.741	976	0	227.300	371.678	544.162	3.874	268915.250
4	22	6	14/02/2018 08:40:23	8804066	14	11	1143	2209	744	0	81.643	203.746	976	0	200.818	362.250	380.733	2.840	366836.083

Merged dataset saved as 'merged_dataset.csv'.
Individual DataFrames cleared from memory.

Merge, Save, and Free Memory output

4 Initial Exploration

> Check missing values and duplicates

▷ 1 cell hidden ...

> Explore target variable distribution

▷ 1 cell hidden ...

Generate Code Markdown

> Update merged dataset

▷ 1 cell hidden ...

> Visualize feature distributions

▷ 1 cell hidden ...

Initial Exploration

4 Initial Exploration

Check missing values and duplicates

```
# Check for missing values
missing_values = merged_df.isnull().sum()
missing_values = missing_values[missing_values > 0]
if missing_values.empty:
    print("No missing values detected in the merged dataset.")
else:
    print("Columns with missing values:")
    display(missing_values.to_frame(name='Missing Count'))

# Check for duplicates
duplicate_count = merged_df.duplicated().sum()
if duplicate_count == 0:
    print("No duplicate rows detected in the merged dataset.")
else:
    print(f"Found {duplicate_count} duplicate rows.")
```

Check missing values and duplicates

Columns with missing values:

	Missing Count
Flow Byts/s	41602
Flow ID	4194300
Src IP	4194300
Src Port	4194300
Dst IP	4194300

Found 396232 duplicate rows.

Check missing values and duplicates output

4 Initial Exploration

› Check missing values and duplicates

▷ 1 cell hidden ...

› Explore target variable distribution

```
# Check if Label column exists
if 'Label' in merged_df.columns:
    label_counts = merged_df['Label'].value_counts()
    label_percentages = merged_df['Label'].value_counts(normalize=True) * 100

    print("Target variable distribution (Label):")
    for label, count in label_counts.items():
        pct = label_percentages[label]
        print(f"{label}: {count} rows ({pct:.2f}%)")
else:
    print("Label column not found in merged dataset.")
```

[9]

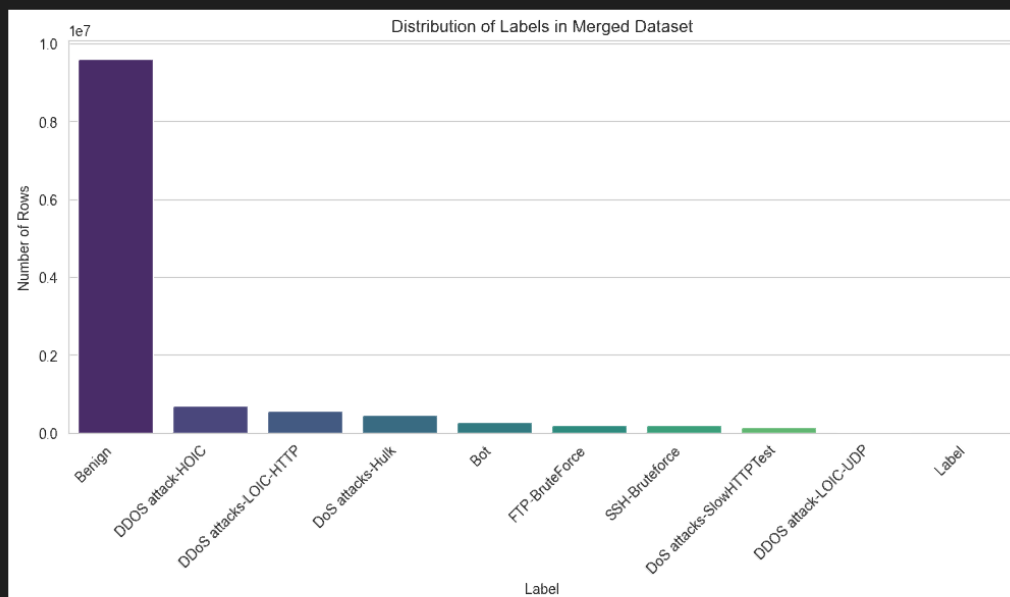
Python

Explore target variable distribution

```
Target variable distribution (Label):
- Benign: 9610172 rows (79.14%)
- DDoS attack-LOIC: 686012 rows (5.65%)
- DDoS attacks-LOIC-HTTP: 576191 rows (4.75%)
- DoS attacks-Hulk: 461912 rows (3.80%)
- Bot: 286191 rows (2.36%)
- FTP-BruteForce: 193360 rows (1.59%)
- SSH-BruteForce: 187589 rows (1.54%)
- DoS attacks-SlowHTTPTest: 139890 rows (1.15%)
- DDoS attack-LOIC-UDP: 1730 rows (0.01%)
- Label: 1 rows (0.00%)
C:\Users\Dokbrac\AppData\Local\Temp\ipykernel_22740\1897861952.py:13: FutureWarning:
```

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the

```
sns.barplot(x=label_counts.index, y=label_counts.values, palette='viridis')
```



Explore target variable distribution output

4 Initial Exploration

> Check missing values and duplicates

▷ 1 cell hidden ...

> Explore target variable distribution

▷ 1 cell hidden ...

√ Update merged dataset

```
# Load the existing merged dataset
merged_df = pd.read_csv("merged_dataset.csv", low_memory=False)

print("Before cleaning:", merged_df.shape)

# Remove bad rows where Label == Label
merged_df = merged_df[merged_df['Label'] != 'Label']

print("After cleaning:", merged_df.shape)

# Overwrite the SAME file (updated dataset)
merged_df.to_csv("merged_dataset.csv", index=False)

print("merged_dataset.csv updated successfully")

# Check labels again
print("\nUpdated Label distribution:")
print(merged_df['Label'].value_counts())
```

[10]

Python

Update merged dataset

```
Before cleaning: (12143048, 84)
After cleaning: (12143047, 84)
merged_dataset.csv updated successfully

Updated Label distribution:
Label
Benign                9610172
DDOS attack-HOIC      686012
DDoS attacks-LOIC-HTTP 576191
DoS attacks-Hulk      461912
Bot                   286191
FTP-BruteForce         193360
SSH-Bruteforce         187589
DoS attacks-SlowHTTPTest 139890
DDOS attack-LOIC-UDP   1730
Name: count, dtype: int64
```

Update merged dataset output

4 Initial Exploration

› Check missing values and duplicates

▷ 1 cell hidden ...

› Explore target variable distribution

▷ 1 cell hidden ...

› Update merged dataset

▷ 1 cell hidden ...

Generate

+ Code

+ Markdown

▶ ✎ ☰ ... 🗑

› Visualize feature distributions

```
# Select numeric columns
numeric_cols = merged_df.select_dtypes(include=np.number).columns.tolist()

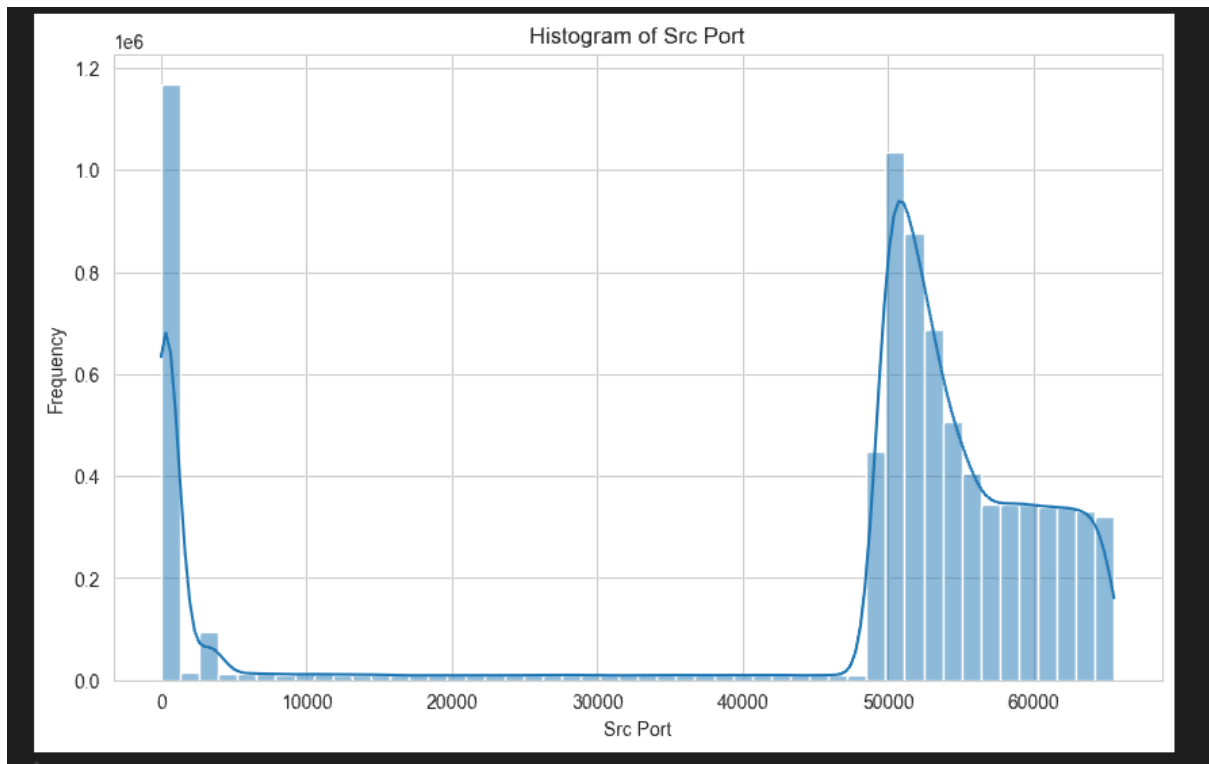
# Plot histograms for the first 10 numeric features
for col in numeric_cols[:10]:
    plt.figure()
    sns.histplot(merged_df[col], bins=50, kde=True)
    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()

print("Histograms plotted for first 10 numeric features.")
```

[11]

Python

Visualize feature distributions



Visualize feature distributions output

5 Early Baseline with Minimal Preprocessing

```
print("Running EARLY baseline with minimal preprocessing")

# Copy dataset (avoid modifying original)
merged_df_baseline = merged_df.copy()

# Remove obvious bad rows
merged_df_baseline = merged_df_baseline[merged_df_baseline['Label'] != 'Label']

# Simple missing handling
merged_df_baseline = merged_df_baseline.fillna(0)

# Encode target ONLY
le_baseline = LabelEncoder()
merged_df_baseline['Label_encoded'] = le_baseline.fit_transform(merged_df_baseline['Label'])

# Use ALL numeric features (NO feature selection)
X_baseline = merged_df_baseline.select_dtypes(include=np.number).drop(columns=['Label_encoded'])
y_baseline = merged_df_baseline['Label_encoded']

print(f"Features used: {X_baseline.shape[1]} (no feature selection)")

# Train-test split
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
    X_baseline, y_baseline,
    test_size=0.3,
    random_state=42,
    stratify=y_baseline
)

print("Train-test split complete")

# Train model
rf_baseline_early = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced"
)

rf_baseline_early.fit(X_train_b, y_train_b)

# Predict
y_pred_b = rf_baseline_early.predict(X_test_b)

# Evaluate
accuracy_b = accuracy_score(y_test_b, y_pred_b)
f1_weighted_b = f1_score(y_test_b, y_pred_b, average='weighted')
f1_macro_b = f1_score(y_test_b, y_pred_b, average='macro')

print("\nEARLY BASELINE RESULTS:")
print(f"Accuracy: {accuracy_b:.4f}")
print(f"F1-weighted: {f1_weighted_b:.4f}")
print(f"F1-macro: {f1_macro_b:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_b, y_pred_b))
```

```

# Evaluate
accuracy_b = accuracy_score(y_test_b, y_pred_b)
f1_weighted_b = f1_score(y_test_b, y_pred_b, average='weighted')
f1_macro_b = f1_score(y_test_b, y_pred_b, average='macro')

print("\nEARLY BASELINE RESULTS:")
print(f"Accuracy: {accuracy_b:.4f}")
print(f"F1-weighted: {f1_weighted_b:.4f}")
print(f"F1-macro: {f1_macro_b:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_b, y_pred_b))

# Save model + results
early_baseline_results = {
    "accuracy": accuracy_b,
    "f1_weighted": f1_weighted_b,
    "f1_macro": f1_macro_b
}

joblib.dump(rf_baseline_early, "rf_early_baseline_model.pkl")
joblib.dump(early_baseline_results, "early_baseline_results.pkl")

print("Early baseline model, results, and encoder saved")

# Clean up RAM
del rf_baseline_early, y_pred_b, early_baseline_results, merged_df_baseline
gc.collect()

print("RAM cleaned after early baseline")

```

Baseline with Minimal Preprocessing

```

Running EARLY baseline with minimal preprocessing
Features used: 1 (no feature selection)
Train-test split complete

EARLY BASELINE RESULTS:
Accuracy: 0.3285
F1-weighted: 0.4169
F1-macro: 0.0865

Classification Report:
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision i
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision i
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Dokbrac\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision i
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

	precision	recall	f1-score	support
0	0.98	0.35	0.51	2883052
1	0.00	0.00	0.00	85857
2	0.00	0.00	0.00	205804
3	0.00	0.00	0.00	519
4	0.11	0.87	0.20	172857
5	0.00	0.00	0.00	138574
6	0.03	1.00	0.06	41967
7	0.00	0.00	0.00	58008
8	0.00	0.00	0.00	56277
accuracy			0.33	3642915
macro avg	0.13	0.25	0.09	3642915
weighted avg	0.78	0.33	0.42	3642915

```

Early baseline model, results, and encoder saved
RAM cleaned after early baseline

```

Baseline with Minimal Preprocessing output

6

Preprocessing

- Handle missing values
 - 1 cell hidden ...
- Encode categorical variables
 - 1 cell hidden ...
- Select relevant features
 - 1 cell hidden ...

Preprocessing

6

Preprocessing

▶

✎

📄

⋮

🗑️

Handle missing values

🔗 Generate

+ Code

+ Markdown

```

# Check missing values
missing_counts = merged_df.isnull().sum()
missing_counts = missing_counts[missing_counts > 0]
print("Columns with missing values before handling:")
display(missing_counts.to_frame('Missing Count'))

# For numeric columns, fill missing with median
numeric_cols = merged_df.select_dtypes(include=np.number).columns
merged_df[numeric_cols] = merged_df[numeric_cols].fillna(merged_df[numeric_cols].median())

# For categorical/object columns, fill missing with mode
categorical_cols = merged_df.select_dtypes(include='object').columns
merged_df[categorical_cols] = merged_df[categorical_cols].fillna(merged_df[categorical_cols].mode().iloc[0])

# Confirm no missing values remain
print("Missing values after handling:")
display(merged_df.isnull().sum().sum())

```

Python

Handle missing values

Columns with missing values before handling:

	Missing Count
Flow Byts/s	41602
Flow ID	4194299
Src IP	4194299
Src Port	4194299
Dst IP	4194299

C:\Users\Dokbrac\AppData\Local\Temp\ipykernel_22740\4053510586.py:12: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes. See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with the new string dtypes.

Missing values after handling:

Handle missing values output

6 Preprocessing

› Handle missing values

▷ 1 cell hidden ...

✓ Encode categorical variables

```
# Initialize LabelEncoder
le = LabelEncoder()

# Fit and transform the Label column
merged_df['Label_encoded'] = le.fit_transform(merged_df['Label'])

# Display the mapping
label_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print("Label encoding completed. Mapping of attacks to integers:")
for label, code in label_mapping.items():
    print(f"  {label} --> {code}")

# Preview encoded column
display(merged_df[['Label', 'Label_encoded']].head())

# Remove extremely rare classes BEFORE split

class_counts = merged_df['Label_encoded'].value_counts()

threshold = 50

rare_classes = class_counts[class_counts < threshold].index

print("Removing rare classes:", rare_classes)

merged_df = merged_df[~merged_df['Label_encoded'].isin(rare_classes)]

print("Shape after removing rare classes:", merged_df.shape)

# Save Encoder
joblib.dump(le, "label_encoder.pkl")
print("Label encoder saved")

print("New class distribution:")
print(merged_df['Label_encoded'].value_counts())
```

[14]

Python

Encode categorical variables

Label encoding completed. Mapping of attacks to integers:

```
Benign --> 0
Bot --> 1
DDoS attack-HOIC --> 2
DDoS attack-LOIC-UDP --> 3
DDoS attacks-LOIC-HTTP --> 4
DoS attacks-Hulk --> 5
DoS attacks-SlowHTTPTest --> 6
FTP-BruteForce --> 7
SSH-BruteForce --> 8
```

	Label	Label_encoded
0	Benign	0
1	Benign	0
2	Benign	0
3	Benign	0
4	Benign	0

Removing rare classes: Index([], dtype='int64', name='Label_encoded')

Shape after removing rare classes: (12143047, 85)

Label encoder saved

New class distribution:

```
Label_encoded
0    9610172
2    686012
4    576191
5    461912
1    286191
7    193360
8    187589
6    139890
3         1730
```

Name: count, dtype: int64

Encode categorical variables output

6 Preprocessing

› Handle missing values

▷ 1 cell hidden ...

› Encode categorical variables

▷ 1 cell hidden ...

~ Select relevant features

Generate Code Markdown

```
# Check for duplicate rows and remove them
duplicate_count = merged_df.duplicated().sum()
print(f"Found {duplicate_count} duplicate rows.")
merged_df = merged_df.drop_duplicates()
print(f"Dataset shape after removing duplicates: {merged_df.shape}")

# Check for constant or zero-variance columns
constant_cols = [col for col in merged_df.columns if merged_df[col].nunique() <= 1]
print(f"Constant/Zero-variance columns: {constant_cols}")
merged_df = merged_df.drop(columns=constant_cols, errors='ignore')
print(f"Dataset shape after dropping constant columns: {merged_df.shape}")

# Select relevant features
selected_features = [
    'Tot Fwd Pkts', 'Tot Bwd Pkts',
    'TotLen Fwd Pkts', 'TotLen Bwd Pkts',
    'Flow Duration', 'SYN Flag Cnt', 'FIN Flag Cnt',
    'RST Flag Cnt', 'PSH Flag Cnt', 'ACK Flag Cnt', 'URG Flag Cnt'
]

# Ensure selected features exist in the dataset
selected_features = [f for f in selected_features if f in merged_df.columns]
print(f"Selected features ({len(selected_features)} columns): {selected_features}")

# Separate features (X) and target (y)
X = merged_df[selected_features]
y = merged_df['Label_encoded']

# Save the cleaned and selected-feature dataset
cleaned_dataset = pd.concat([X, y], axis=1)
cleaned_dataset.to_csv("cleaned_selected_features_dataset.csv", index=False)
print("Cleaned dataset saved as 'cleaned_selected_features_dataset.csv'")

# Preview
display(cleaned_dataset.head())
```

[15]

Python

Select relevant features

```
Found 396232 duplicate rows.
Dataset shape after removing duplicates: (11746815, 85)
Constant/Zero-variance columns: ['Bwd PSH Flags', 'Fwd URG Flags', 'Bwd URG Flags', 'CWE Flag Count', 'Fwd Byts/b Avg', 'Fwd Pkts/b Avg', 'Fwd Blk Rate']
Dataset shape after dropping constant columns: (11746815, 75)
Selected features (11 columns): ['Tot Fwd Pkts', 'Tot Bwd Pkts', 'TotLen Fwd Pkts', 'TotLen Bwd Pkts', 'Flow Duration', 'SYN Flag Cnt', 'FIN Flag Cnt', 'RST Flag Cnt', 'PSH Flag Cnt', 'ACK Flag Cnt', 'URG Flag Cnt']
Cleaned dataset saved as 'cleaned_selected_features_dataset.csv'
```

	Tot Fwd Pkts	Tot Bwd Pkts	TotLen Fwd Pkts	TotLen Bwd Pkts	Flow Duration	SYN Flag Cnt	FIN Flag Cnt	RST Flag Cnt	PSH Flag Cnt	ACK Flag Cnt	URG Flag Cnt	Label_encoded
0	3	0	0	0	112641719	0	0	0	0	0	0	0
1	3	0	0	0	112641466	0	0	0	0	0	0	0
2	3	0	0	0	112638623	0	0	0	0	0	0	0
3	15	10	1239	2273	6453966	0	0	0	1	0	0	0
4	14	11	1143	2209	8804066	0	0	0	1	0	0	0

Select relevant features output

7 Train-Test Split

Stratified Train-Test Split with Rare Class Removal

▷ 1 cell hidden ...

Label Check

▷ 1 cell hidden ...

Train-Test Split

7 Train-Test Split

Stratified Train-Test Split with Rare Class Removal

```
# Load cleaned dataset
merged_df = pd.read_csv(
    "cleaned_selected_features_dataset.csv",
    low_memory=False
)

# Force numeric conversion
for col in merged_df.columns:
    if col != "Label_encoded":
        merged_df[col] = pd.to_numeric(merged_df[col], errors='coerce')

# Fill any NaNs created
merged_df.fillna(merged_df.median(numeric_only=True), inplace=True)
print(f"Cleaned dataset loaded. Shape: {merged_df.shape}")

# Check class distribution
class_counts = merged_df['Label_encoded'].value_counts()
print("Label distribution before removing rare classes:")
print(class_counts)

# Remove classes with fewer than 2 samples
rare_classes = class_counts[class_counts < 2].index.tolist()
print(f"Removing rare classes with < 2 samples: {rare_classes}")
merged_df = merged_df[~merged_df['Label_encoded'].isin(rare_classes)]
print(f"Dataset shape after removing rare classes: {merged_df.shape}")

# Clean Train-Test Split

X = merged_df[selected_features]
y = merged_df['Label_encoded']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    stratify=y,
    random_state=42
)

print("Train-test split completed")
print("X_train:", X_train.shape)
print("y_train distribution:\n", y_train.value_counts())

# Remove very rare classes from Training Set Only

class_counts = y_train.value_counts()

threshold = 20

rare_classes = class_counts[class_counts < threshold].index
```

[16]

Python

```

# Clean Train-Test Split

X = merged_df[selected_features]
y = merged_df['Label_encoded']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    stratify=y,
    random_state=42
)

print("Train-test split completed")
print("X_train:", X_train.shape)
print("y_train distribution:\n", y_train.value_counts())

# Remove very rare classes from Training Set Only

class_counts = y_train.value_counts()

threshold = 20

rare_classes = class_counts[class_counts < threshold].index

print("Removing rare classes:", rare_classes)

mask = ~y_train.isin(rare_classes)

X_train = X_train[mask]
y_train = y_train[mask]

print("New training shape:", X_train.shape)
print("New training distribution:\n", y_train.value_counts())

```

Python

Stratified Train-Test Split with Rare Class Removal

```

Removing rare classes with < 2 samples: []
Dataset shape after removing rare classes: (11746815, 12)
Train-test split completed
X_train: (8222770, 11)
y_train distribution:
Label_encoded
0    6724979
2    467923
4    403334
5    304411
1    197617
8     82125
7     27546
6    13624
3     1211
Name: count, dtype: int64
Removing rare classes: Index([], dtype='int64', name='Label_encoded')
New training shape: (8222770, 11)
New training distribution:
Label_encoded
0    6724979
2    467923
4    403334
5    304411
1    197617
8     82125
7     27546
6    13624
3     1211
Name: count, dtype: int64

```

Stratified Train-Test Split with Rare Class Removal output

7 Train-Test Split

> Stratified Train-Test Split with Rare Class Removal

▷ 1 cell hidden ...

Label Check

Generate Code Markdown

```
print("Encoded distribution:")
print(merged_df['Label_encoded'].value_counts())
print(y_train.value_counts())
print(set(y_train))
```

[17]

Python

Label Check

```
Encoded distribution:
Label_encoded
0    9607114
2     668461
4     576191
5     434873
1     282310
8     117322
7      39352
6      19462
3       1730
Name: count, dtype: int64
Label_encoded
0    6724979
2     467923
4     403334
5     304411
1     197617
8      82125
7      27546
6      13624
3       1211
Name: count, dtype: int64
{0, 1, 2, 3, 4, 5, 6, 7, 8}
```

Label Check output

8 Training

> Inspect class distribution in training data

▷ 1 cell hidden ...

> SMOTE(SAMPLING)

▷ 1 cell hidden ...

> SMOTE(SAMPLING) + HYPERPARAMETER TUNING

▷ 1 cell hidden ...

> BORDERLINE SMOTE(SAMPLING)

▷ 1 cell hidden ...

> BORDERLINE SMOTE(SAMPLING) + HYPERPARAMETER TUNING

▷ 1 cell hidden ...

Training

8 Training

~ Inspect class distribution in training data

```
# Check distribution in y_train
class_counts_train = y_train.value_counts()
print("Training set label distribution before resampling:")
print(class_counts_train)
```

[18]

Python

Inspect class distribution in training data

```
Training set label distribution before resampling:
Label_encoded
0      6724979
2      467923
4      403334
5      304411
1      197617
8       82125
7       27546
6       13624
3        1211
Name: count, dtype: int64
```

Inspect class distribution in training data output

8 Training

› Inspect class distribution in training data

▷ 1 cell hidden ...



√ SMOTE(SAMPLING)

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    ...
    ...target = int(count * 2)
    ...target = min(target, max_limit)
    ...target = max(target, count)
    ...target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# Apply SMOTE
smote = SMOTE(sampling_strategy=target_samples, random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Class distribution AFTER SMOTE:")
print(pd.Series(y_train_smote).value_counts().sort_index())

# Train Random Forest
rf_smote = RandomForestClassifier(
    ...n_estimators=100,
    ...random_state=42,
    ...n_jobs=-1,
    ...class_weight=None
)
rf_smote.fit(X_train_smote, y_train_smote)

# Save model
joblib.dump(rf_smote, "rf_smote_model.pkl")
print("SMOTE Random Forest trained and saved")

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Predicted classes:", set(y_pred_smote))

# Evaluate
accuracy = accuracy_score(y_test, y_pred_smote)
```

[19]

Python

```

# Save model
joblib.dump(rf_smote, "rf_smote_model.pkl")
print("SMOTE Random Forest trained and saved")

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Predicted classes:", set(y_pred_smote))

# Evaluate
accuracy = accuracy_score(y_test, y_pred_smote)
f1_weighted = f1_score(y_test, y_pred_smote, average='weighted')
f1_macro = f1_score(y_test, y_pred_smote, average='macro')

print("\nSMOTE Model Evaluation:")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred_smote))

# Save results
smote_results = {
    "accuracy": accuracy,
    "f1_weighted": f1_weighted,
    "f1_macro": f1_macro
}
joblib.dump(smote_results, "smote_results.pkl")
print("SMOTE results saved")

# 8: Clean up memory
del smote, X_train_smote, y_train_smote, rf_smote, y_pred_smote, smote_results
gc.collect()
print("RAM cleaned after SMOTE")

```

Python

SMOTE(SAMPLING)

```

7      55092
6      27248
3      2422
Name: count, dtype: int64
Predicted classes: {np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)}

SMOTE Model Evaluation:
Accuracy: 0.9628
F1-weighted: 0.9616
F1-macro: 0.8140

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.86	0.99	0.92	84693
2	0.72	0.86	0.79	200538
3	0.78	0.92	0.85	519
4	0.98	0.98	0.98	172857
5	0.70	0.48	0.57	130462
6	0.66	0.40	0.50	5838
7	0.72	0.92	0.81	11806
8	0.91	0.95	0.93	35197
accuracy			0.96	3524045
macro avg	0.81	0.83	0.81	3524045
weighted avg	0.96	0.96	0.96	3524045

```

SMOTE results saved
RAM cleaned after SMOTE

```

SMOTE(SAMPLING) output

8 Training

Inspect class distribution in training data

▷ 1 cell hidden ...

SMOTE(SAMPLING)

▷ 1 cell hidden ...

SMOTE(SAMPLING) + HYPERPARAMETER TUNING

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    target = int(count * 2)
    target = min(target, max_limit)
    target = max(target, count)
    target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# SMOTE Hyperparameter Tuning

best_f1 = 0
best_k = None
best_model = None
best_y_pred = None

for k in [3, 5, 7, 9]:
    print(f"\nTesting SMOTE k_neighbors = {k}")

    smote = SMOTE(
        sampling_strategy=target_samples,
        k_neighbors=k,
        random_state=42
    )

    X_res, y_res = smote.fit_resample(X_train, y_train)

    rf = RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        . . .
```

88]

Python

```

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    target = int(count * 2)
    target = min(target, max_limit)
    target = max(target, count)
    target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# SMOTE Hyperparameter Tuning

best_f1 = 0
best_k = None
best_model = None
best_y_pred = None

for k in [3, 5, 7, 9]:
    print(f"\nTesting SMOTE k_neighbors = {k}")

    smote = SMOTE(
        sampling_strategy=target_samples,
        k_neighbors=k,
        random_state=42
    )

    X_res, y_res = smote.fit_resample(X_train, y_train)

    rf = RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1,
        class_weight=None
    )

    rf.fit(X_res, y_res)

    y_pred = rf.predict(X_test)

    print("SMOTE(HYPERPARAMETER) distribution:\n", pd.Series(y_res).value_counts())
    print("Predicted classes:", set(y_pred))

    f1 = f1_score(y_test, y_pred, average='macro')

    print(f"F1-macro: {f1:.4f}")

    if f1 > best_f1:
        best_f1 = f1
        best_k = k
        best_model = rf
        best_y_pred = y_pred

# Save Best Model

joblib.dump(best_model, "rf_smote_tuned_model.pkl")

```

Python

```

# Save Best Model

joblib.dump(best_model, "rf_smote_tuned_model.pkl")
print("\nBEST MODEL SAVED")

print(f"Best k_neighbors: {best_k}")
print(f"Best F1-macro: {best_f1:.4f}")

# Final Evaluation

accuracy = accuracy_score(y_test, best_y_pred)
f1_weighted = f1_score(y_test, best_y_pred, average='weighted')
f1_macro = f1_score(y_test, best_y_pred, average='macro')

print("\nFINAL RESULTS")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, best_y_pred))

# Save Results

smote_results = {
    # "best_k": best_k,
    "accuracy": accuracy,
    "f1_weighted": f1_weighted,
    "f1_macro": f1_macro
}

joblib.dump(smote_results, "smote_tuned_results.pkl")
print("Results saved")

# Clean Memory

del smote, X_res, y_res, rf, best_model, best_y_pred, smote_results
gc.collect()

print("RAM cleaned")

```

Python

SMOTE(SAMPLING) + HYPERPARAMETER TUNING

```

F1-macro: 0.8146

BEST MODEL SAVED
Best k_neighbors: 3
Best F1-macro: 0.8154

FINAL RESULTS
Accuracy: 0.9628
F1-weighted: 0.9616
F1-macro: 0.8154

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.86	0.99	0.92	84693
2	0.72	0.86	0.79	200538
3	0.79	0.94	0.85	519
4	0.98	0.98	0.98	172857
5	0.70	0.48	0.57	130462
6	0.66	0.40	0.50	5838
7	0.72	0.92	0.81	11806
8	0.91	0.95	0.93	35197
accuracy			0.96	3524045
macro avg	0.81	0.83	0.82	3524045
weighted avg	0.96	0.96	0.96	3524045

```

Results saved
RAM cleaned

```

SMOTE(SAMPLING) + HYPERPARAMETER TUNING output

8 Training

› Inspect class distribution in training data

▷ 1 cell hidden ...

› SMOTE(SAMPLING)

▷ 1 cell hidden ...

› SMOTE(SAMPLING) + HYPERPARAMETER TUNING

▷ 1 cell hidden ...



› BORDERLINE SMOTE(SAMPLING)

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    target = int(count * 2)
    target = min(target, max_limit)
    target = max(target, count)
    target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# Apply Borderline-SMOTE
smote = BorderlineSMOTE(
    sampling_strategy=target_samples,
    random_state=42,
    kind='borderline-1'
)

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Class distribution AFTER Borderline-SMOTE:")
print(pd.Series(y_train_smote).value_counts())

# Train Random Forest
rf_smote = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1,
    class_weight=None
```

[21]

Python

```

# Apply Borderline-SMOTE
smote = BorderlineSMOTE(
    sampling_strategy=target_samples,
    random_state=42,
    kind='borderline-1'
)

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Class distribution AFTER Borderline-SMOTE:")
print(pd.Series(y_train_smote).value_counts())

# Train Random Forest
rf_smote = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1,
    class_weight=None
)
rf_smote.fit(X_train_smote, y_train_smote)

# Step 4: Save model
joblib.dump(rf_smote, "rf_borderline_smote_model.pkl")
print("Borderline-SMOTE Random Forest trained and saved")

# Predict on original test set
y_pred_smote = rf_smote.predict(X_test)

print("BORDERLINE_SMOTE distribution:\n", pd.Series(y_train_smote).value_counts())
print("Unique classes in prediction:", len(set(y_pred_smote)))

# Evaluate
accuracy = accuracy_score(y_test, y_pred_smote)
f1_weighted = f1_score(y_test, y_pred_smote, average='weighted')
f1_macro = f1_score(y_test, y_pred_smote, average='macro')

print("\nBorderline-SMOTE Model Evaluation:")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_smote))

# Step 7: Save results
smote_results = {
    "accuracy": accuracy,
    "f1_weighted": f1_weighted,
    "f1_macro": f1_macro
}
joblib.dump(smote_results, "borderline_smote_results.pkl")
print("Borderline-SMOTE results saved")

# Clean up memory
del smote, X_train_smote, y_train_smote, rf_smote, y_pred_smote, smote_results
gc.collect()
print("RAM cleaned after Borderline-SMOTE")

```

Python

BORDERLINE SMOTE(SAMPLING)

```

7      55092
6      27248
3       2422
Name: count, dtype: int64
Unique classes in prediction: 9

Borderline-SMOTE Model Evaluation:
Accuracy: 0.9598
F1-weighted: 0.9588
F1-macro: 0.7794

Classification Report:
      precision    recall  f1-score   support

0         1.00      0.99      0.99    2882135
1         0.83      0.99      0.91     84693
2         0.73      0.85      0.78    200538
3         0.75      0.95      0.84      519
4         0.98      0.98      0.98    172857
5         0.69      0.48      0.57    130462
6         0.42      0.34      0.38      5838
7         0.67      0.78      0.73     11806
8         0.78      0.93      0.85     35197

 accuracy          0.96    3524045
 macro avg         0.76    0.81    0.78    3524045
 weighted avg      0.96    0.96    0.96    3524045

Borderline-SMOTE results saved
RAM cleaned after Borderline-SMOTE

```

BORDERLINE SMOTE(SAMPLING) output

8 Training

› Inspect class distribution in training data

▷ 1 cell hidden ...

› SMOTE(SAMPLING)

▷ 1 cell hidden ...

› SMOTE(SAMPLING) + HYPERPARAMETER TUNING

▷ 1 cell hidden ...

› BORDERLINE SMOTE(SAMPLING)

▷ 1 cell hidden ...

Generate Code Markdown

Play Edit View ... Delete

› BORDERLINE SMOTE(SAMPLING) + HYPERPARAMETER TUNING

```
# Analyze class distribution
class_counts = y_train.value_counts()
print("Original distribution:")
print(class_counts)

# Target sampling

majority = class_counts.max()
max_limit = int(majority * 0.05)

target_samples = {}

for cls, count in class_counts.items():
    target = int(count * 2)
    target = min(target, max_limit)
    target = max(target, count)
    target_samples[cls] = target

print("Target sampling strategy:")
print(target_samples)

# BORDERLINE-SMOTE hyperparameter Tuning

best_f1 = 0
best_params = None
best_model = None
best_pred = None

for k in [3, 5, 7]:
    for m in [10, 20]:
        print(f"Testing k neighbors (k) = neighbors (m)")
```

[1

Python

```

# BORDERLINE-SMOTE hyperparameter Tuning

best_f1 = 0
best_params = None
best_model = None
best_pred = None

for k in [3, 5, 7]:
    for m in [10, 20]:

        print(f"\nTesting k_neighbors={k}, m_neighbors={m}")

        smote = BorderlineSMOTE(
            sampling_strategy=target_samples,
            k_neighbors=k,
            m_neighbors=m,
            kind='borderline-1',
            random_state=42
        )

        X_res, y_res = smote.fit_resample(X_train, y_train)

        rf = RandomForestClassifier(
            n_estimators=100,
            random_state=42,
            n_jobs=-1
        )

        rf.fit(X_res, y_res)

        y_pred = rf.predict(X_test)

        print("Borderline_SMOTE(HYPERPARAMETER) distribution:\n", pd.Series(y_res).value_counts())
        print("Unique predicted classes:", len(set(y_pred)))

        f1 = f1_score(y_test, y_pred, average='macro')

        print(f"F1-macro: {f1:.4f}")

        if f1 > best_f1:
            best_f1 = f1
            best_params = (k, m)
            best_model = rf
            best_pred = y_pred

# Save best Model

joblib.dump(best_model, "rf_borderline_smote_tuned_model.pkl")

print("\nBEST BORDERLINE-SMOTE MODEL")
print(f"k_neighbors: {best_params[0]}")
print(f"m_neighbors: {best_params[1]}")
print(f"Best F1-macro: {best_f1:.4f}")

# Evaluation

accuracy = accuracy_score(y_test, best_pred)
f1_weighted = f1_score(y_test, best_pred, average='weighted')
f1_macro = f1_score(y_test, best_pred, average='macro')

print("\nFINAL RESULTS")

```

Python

```

# Save best Model

joblib.dump(best_model, "rf_borderline_smote_tuned_model.pkl")

print("\nBEST BORDERLINE-SMOTE MODEL")
print(f"k_neighbors: {best_params[0]}")
print(f"m_neighbors: {best_params[1]}")
print(f"Best F1-macro: {best_f1:.4f}")

# Evaluation

accuracy = accuracy_score(y_test, best_pred)
f1_weighted = f1_score(y_test, best_pred, average='weighted')
f1_macro = f1_score(y_test, best_pred, average='macro')

print("\nFINAL RESULTS")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-weighted: {f1_weighted:.4f}")
print(f"F1-macro: {f1_macro:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, best_pred))

# Save Results

results = {
    # "best_k_neighbors": best_params[0],
    # "best_m_neighbors": best_params[1],
    "accuracy": accuracy,
    "f1_weighted": f1_weighted,
    "f1_macro": f1_macro
}

joblib.dump(results, "borderline_smote_tuned_results.pkl")

# Clean Memory

del smote, X_res, y_res, rf, best_model, best_pred, results
gc.collect()

print("DONE")

```

Python

BORDERLINE SMOTE(SAMPLING) + HYPERPARAMETER TUNING

```

BEST BORDERLINE-SMOTE MODEL
k_neighbors: 3
m_neighbors: 20
Best F1-macro: 0.7808

FINAL RESULTS
Accuracy: 0.9603
F1-weighted: 0.9591
F1-macro: 0.7808

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	2882135
1	0.82	1.00	0.90	84693
2	0.73	0.85	0.78	200538
3	0.76	0.93	0.84	519
4	0.98	0.98	0.98	172857
5	0.70	0.48	0.57	130462
6	0.40	0.39	0.39	5838
7	0.67	0.73	0.70	11806
8	0.83	0.92	0.88	35197
accuracy			0.96	3524045
macro avg	0.76	0.81	0.78	3524045
weighted avg	0.96	0.96	0.96	3524045

```

DONE

```

BORDERLINE SMOTE(SAMPLING) + HYPERPARAMETER TUNING output

9 Model Evaluation

› Load all results

▷ 1 cell hidden ...

› Create comparison table

▷ 1 cell hidden ...

› Visual comparison

▷ 1 cell hidden ...

› Automatically select best model

▷ 1 cell hidden ...

› Select + save final model

▷ 1 cell hidden ...

› Add Confusion Matrix

▷ 1 cell hidden ...

› Test real predictions

▷ 1 cell hidden ...

› Save ALL final artifacts

▷ 1 cell hidden ...

Model Evaluation

9 Model Evaluation

Load all results

```
early_baseline = joblib.load("early_baseline_results.pkl")

smote = joblib.load("smote_results.pkl")
smote_hyperparameter = joblib.load("smote_tuned_results.pkl")

borderline = joblib.load("borderline_smote_results.pkl")
borderline_hyperparameter = joblib.load("borderline_smote_tuned_results.pkl")
```

[23]

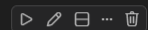
Python

Load all results

9 Model Evaluation

> Load all results

▷ 1 cell hidden ...



▼ Create comparison table

```
results_df = pd.DataFrame([
    early_baseline,
    smote,
    smote_hyperparameter,
    borderline,
    borderline_hyperparameter
], index=["Early Baseline",
        "SMOTE",
        "SMOTE(Hyperparameter)",
        "Borderline-SMOTE",
        "Borderline-SMOTE(Hyperparameter)"])

print("Model Comparison Table")
display(results_df)
```

[24]

Python

Create comparison table

Model Comparison Table				
	accuracy	f1_weighted	f1_macro	
Early Baseline	0.329	0.417	0.087	
SMOTE	0.963	0.962	0.814	
SMOTE(Hyperparameter)	0.963	0.962	0.815	
Borderline-SMOTE	0.960	0.959	0.779	
Borderline-SMOTE(Hyperparameter)	0.960	0.959	0.781	

Create comparison table output

9 Model Evaluation

> Load all results

▷ 1 cell hidden ...

> Create comparison table

▷ 1 cell hidden ...

Visual comparison

```
results_df.plot(kind="bar", figsize=(10,6))

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.legend(loc="lower right")

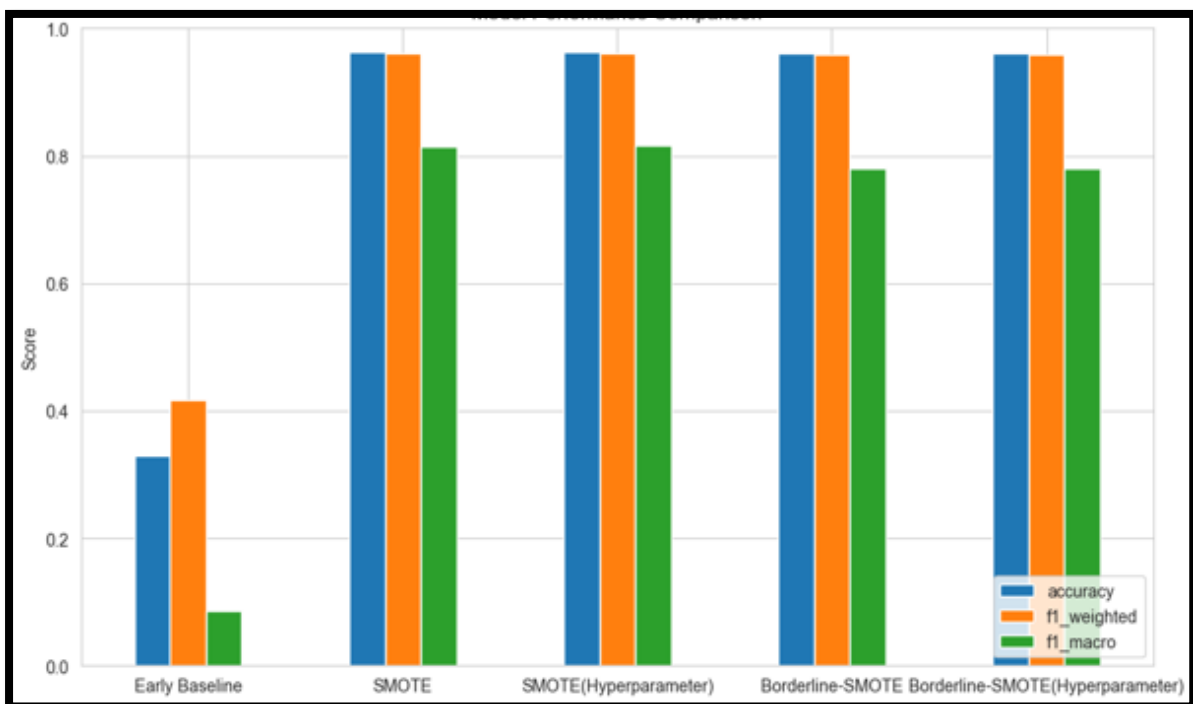
plt.tight_layout()

plt.show()
```

[25]

Python

Visual comparison



Visual comparison output

9 Model Evaluation

- Load all results
 - 1 cell hidden ...
- Create comparison table
 - 1 cell hidden ...
- Visual comparison
 - 1 cell hidden ...
- Automatically select best model

```
best_model_name = results_df["f1_macro"].idxmax()
print("Best Model:", best_model_name)
```

[26] Python

Automatically select best model

```
Best Model: SMOTE(Hyperparameter)
```

Automatically select best model output

9 Model Evaluation

- Load all results
 - 1 cell hidden ...
- Create comparison table
 - 1 cell hidden ...
- Visual comparison
 - 1 cell hidden ...
- Automatically select best model
 - 1 cell hidden ...
- Select + save final model

```
model_map = {
    "Early Baseline": "rf_early_baseline_model.pkl",
    "SMOTE": "rf_smote_model.pkl",
    "SMOTE(Hyperparameter)": "rf_smote_tuned_model.pkl",
    "Borderline-SMOTE": "rf_borderline_smote_model.pkl",
    "Borderline-SMOTE(Hyperparameter)": "rf_borderline_smote_tuned_model.pkl"
}

best_model = joblib.load(model_map[best_model_name])
joblib.dump(best_model, "final_nids_model.pkl")
```

[27] Python

Select + save final model

```
['final_nids_model.pkl']
```

Select + save final model output

9

Model Evaluation

GenerateCodeMarkdown

>

Load all results

1 cell hidden

>

Create comparison table

1 cell hidden

>

Visual comparison

1 cell hidden

>

Automatically select best model

1 cell hidden

>

Select + save final model

1 cell hidden

>

Add Confusion Matrix

1 cell hidden

Python

```
y_pred = best_model.predict(X_test)

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10,8))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")

plt.tight_layout()
plt.show()
```

Add Confusion Matrix



Add Confusion Matrix output

9 Model Evaluation

> Load all results

▶ 1 cell hidden ...

> Create comparison table

▶ 1 cell hidden ...

> Visual comparison

▶ 1 cell hidden ...

> Automatically select best model

▶ 1 cell hidden ...

> Select + save final model

▶ 1 cell hidden ...

> Add Confusion Matrix

▶ 1 cell hidden ...

▼ Test real predictions

```
best_model = joblib.load("final_nids_model.pkl")
le = joblib.load("label_encoder.pkl")

sample = X_test.iloc[[0]]

pred = best_model.predict(sample)[0]
true = y_test.iloc[0]

print("True label (encoded):", true)
print("Predicted label (encoded):", pred)

print("True label (text):", le.inverse_transform([true])[0])
print("Predicted label (text):", le.inverse_transform([pred])[0])
```

[29]

Python

Test real predictions

```
True label (encoded): 0
Predicted label (encoded): 0
True label (text): Benign
Predicted label (text): Benign
```

Test real predictions output

9 Model Evaluation

> Load all results

▷ 1 cell hidden ...

> Create comparison table

▷ 1 cell hidden ...

> Visual comparison

▷ 1 cell hidden ...

> Automatically select best model

▷ 1 cell hidden ...

> Select + save final model

▷ 1 cell hidden ...

> Add Confusion Matrix

▷ 1 cell hidden ...

> Test real predictions

▷ 1 cell hidden ...

✓ Save ALL final artifacts

```
# Save final artifacts
joblib.dump(best_model, "final_nids_model.pkl")
joblib.dump(le, "label_encoder.pkl")
joblib.dump(selected_features, "features.pkl")

print("Final model + encoder + features saved")

# Cleanup
del best_model, le, selected_features
gc.collect()
print("Memory cleared")
```

[30]

Python

Save ALL final artifacts

```
Final model + encoder + features saved
Memory cleared
```

Save ALL final artifacts output